Week 6 – Parallelism Part 2

02613 Python and High-Performance Computing

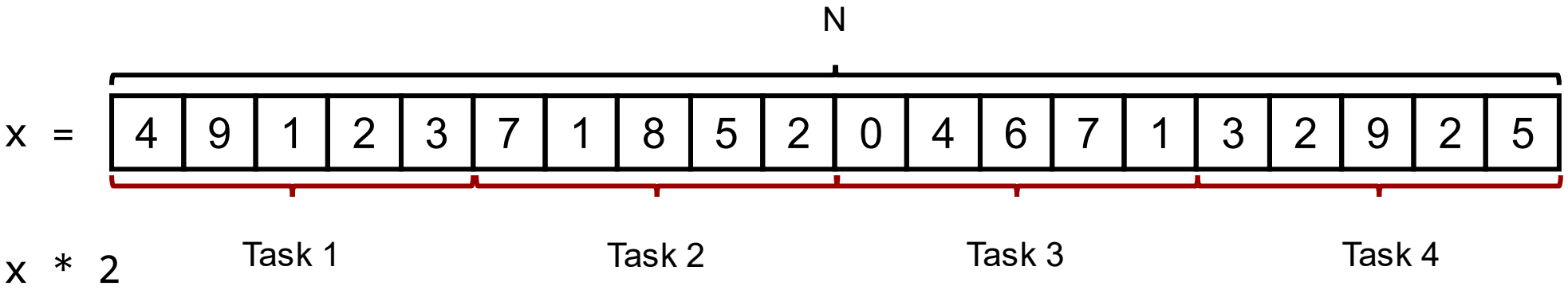
Today

- Parallel reductions

- Hardware interferes again!

Reductions

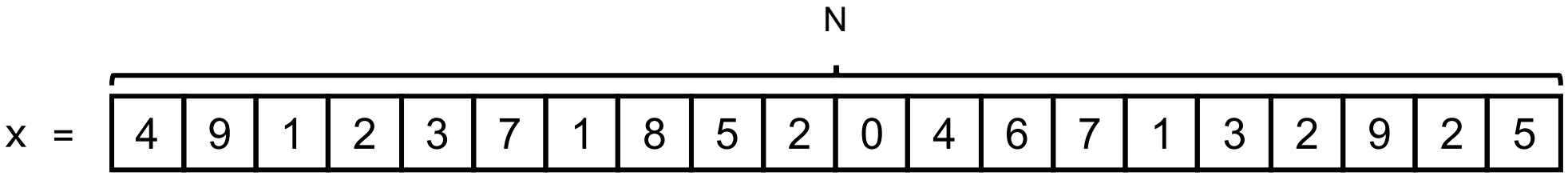
Recall



Normally	N time units
2 Processors	N / 2 time units
4 Processors	N / 4 time units

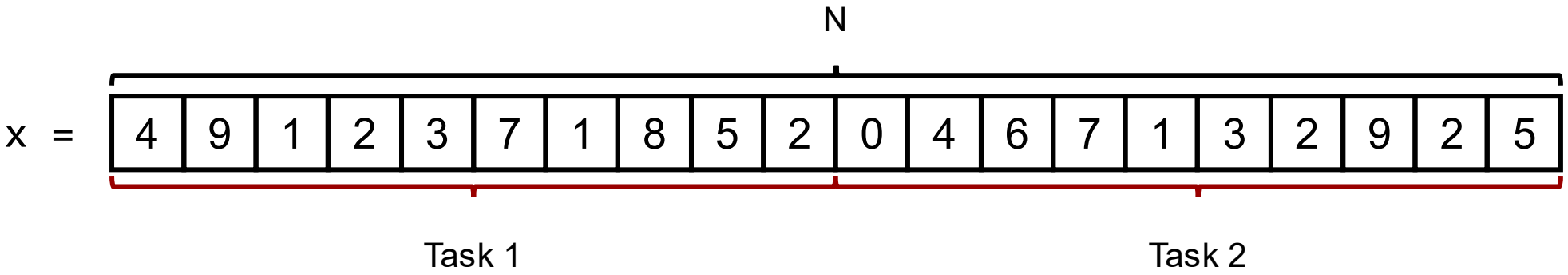
Embarrassingly parallel:
Every unit of work is independent.
I.e., everything can be parallelized.

Motivation: parallel sum




```
sum(x)
```

Motivation: parallel sum

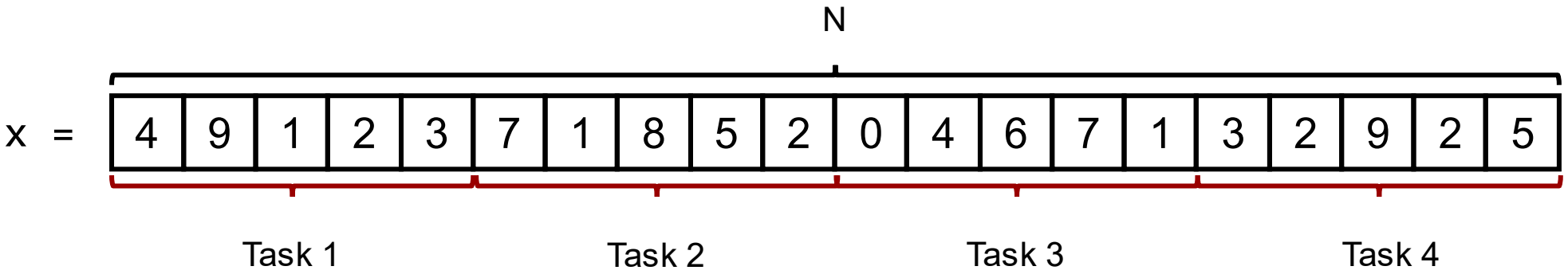



```
sum(x)
```

```
sum(x[:N/2]) + sum(x[N/2:])
```

Task 1 Task 2

Motivation: parallel sum

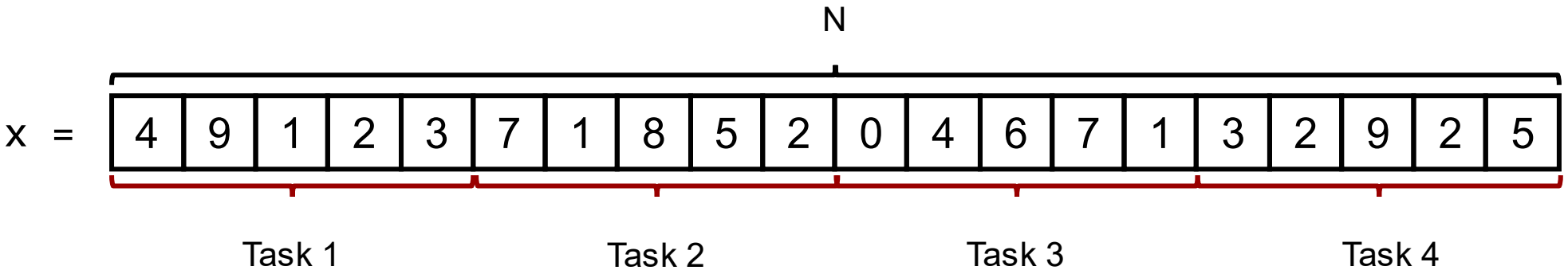



```
sum(x)
```

```
sum(x[:N/4]) + sum(x[N/4:2*N/4]) + sum(x[2*N/4:3*N/4]) + sum(x[3*N/2:])
```

Task 1 Task 2 Task 3 Task 4

Motivation: parallel sum

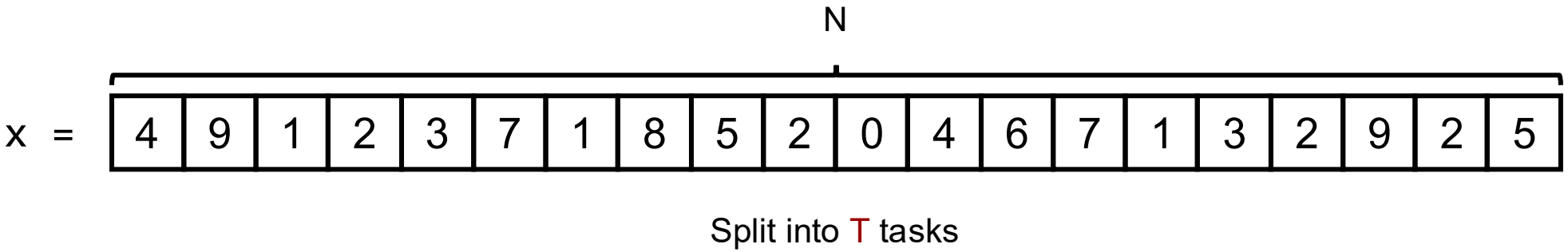



```
sum(x)
```

```
sum(x[:N/4]) + sum(x[N/4:2*N/4]) + sum(x[2*N/4:3*N/4]) + sum(x[3*N/2:])
```

Task 1 Task 2 Task 3 Task 4

Motivation: parallel sum




```
sum(x)
```

```
sum(x[:N/T]) + sum(x[N/T:2*N/T]) + sum(x[2*N/T:3*N/T]) + ...
```

Each task: N/T operations

Final sum: T operations

Motivation: parallel sum

N

x = | 4 | 9 | 1 | 2 | 3 | 7 | 1 | 8 | 5 | 2 | 0 | 4 | 6 | 7 | 1 | 3 | 2 | 9 | 2 | 5 |

Split into T tasks

```
sum(x)
```

```
sum(x[:N/T]) + sum(x[N/T:2*N/T]) + sum(x[2*N/T:3*N/T]) + ...
```

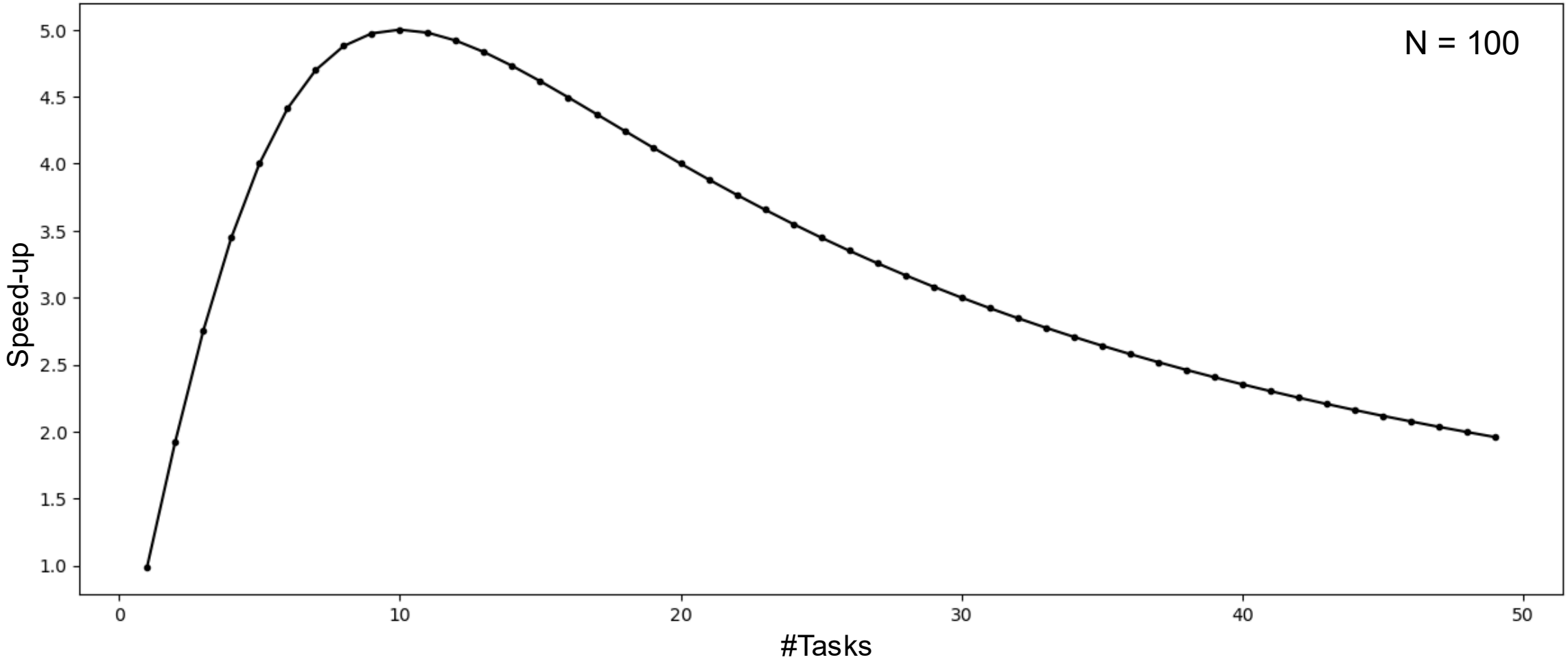
Each task: N/T operations

Total: N/T + T

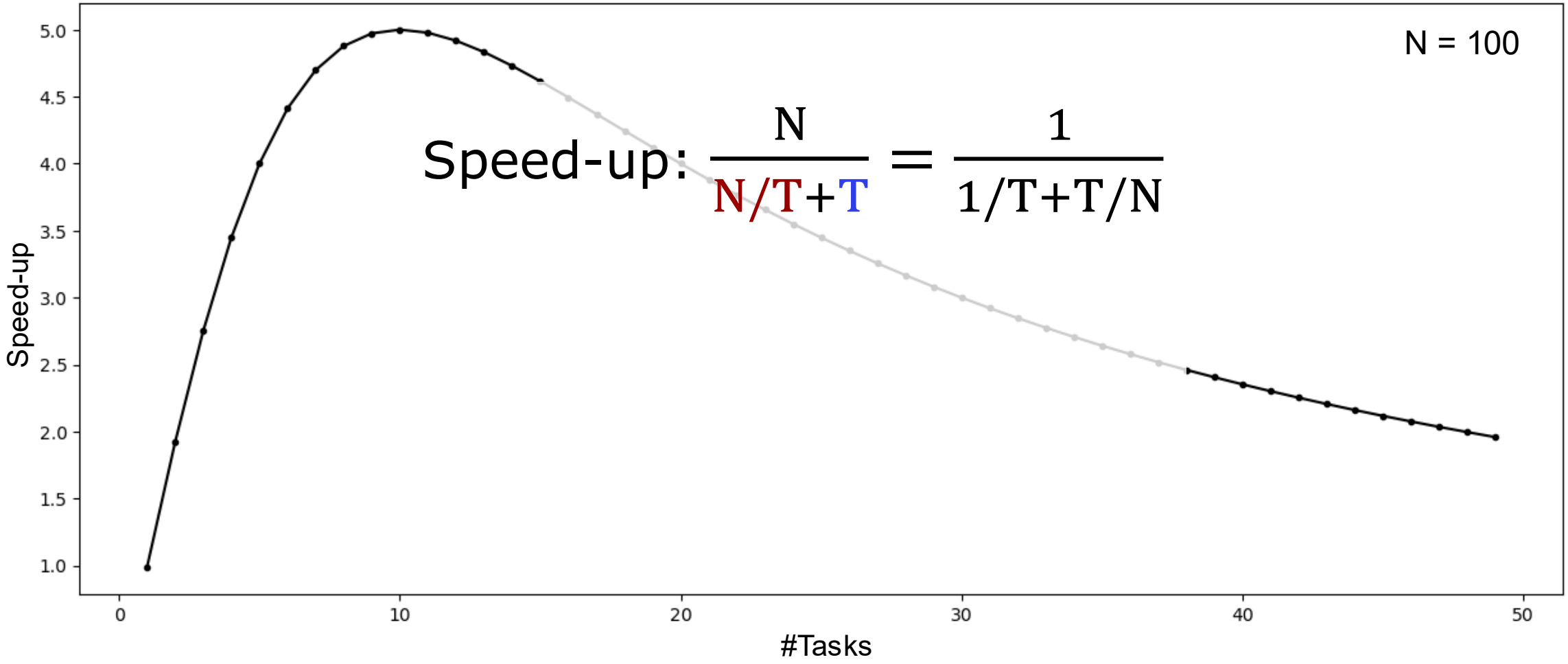
Final sum: T operations

Speed-up: $\frac{N}{N/T+T} = \frac{1}{1/T+T/N}$

Motivation: parallel sum

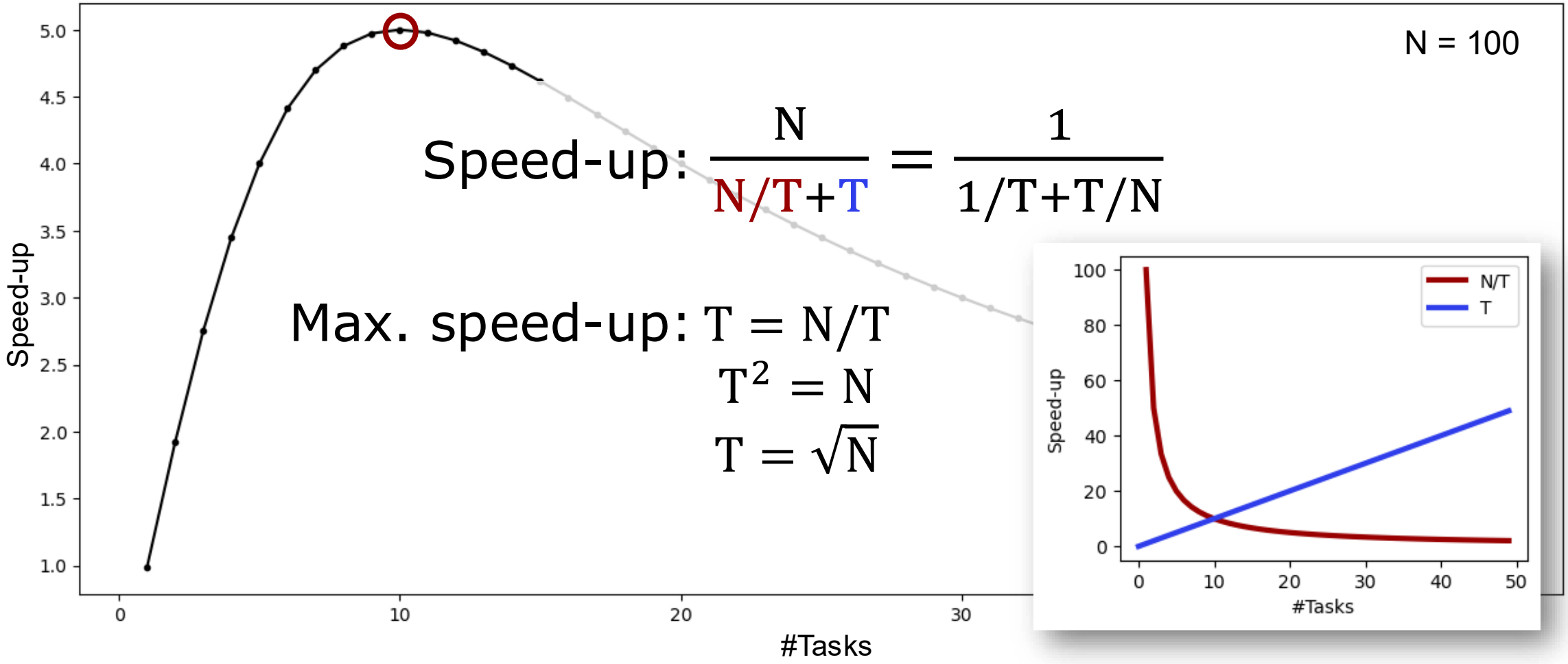


Motivation: parallel sum



$$\text{Speed-up: } \frac{N}{N/T+T} = \frac{1}{1/T+T/N}$$

Motivation: parallel sum



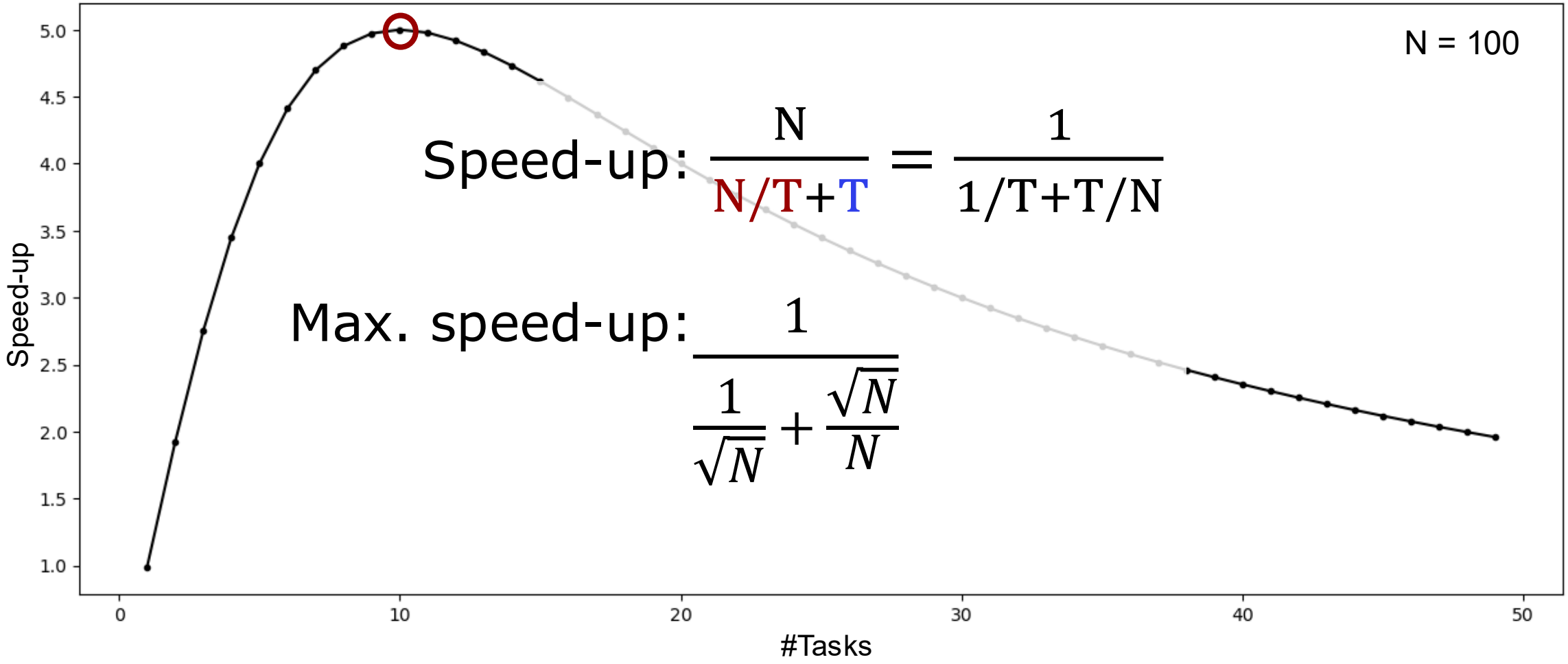
Speed-up: $\frac{N}{N/T+T} = \frac{1}{1/T+T/N}$

Max. speed-up: $T = N/T$

$T^2 = N$

$T = \sqrt{N}$

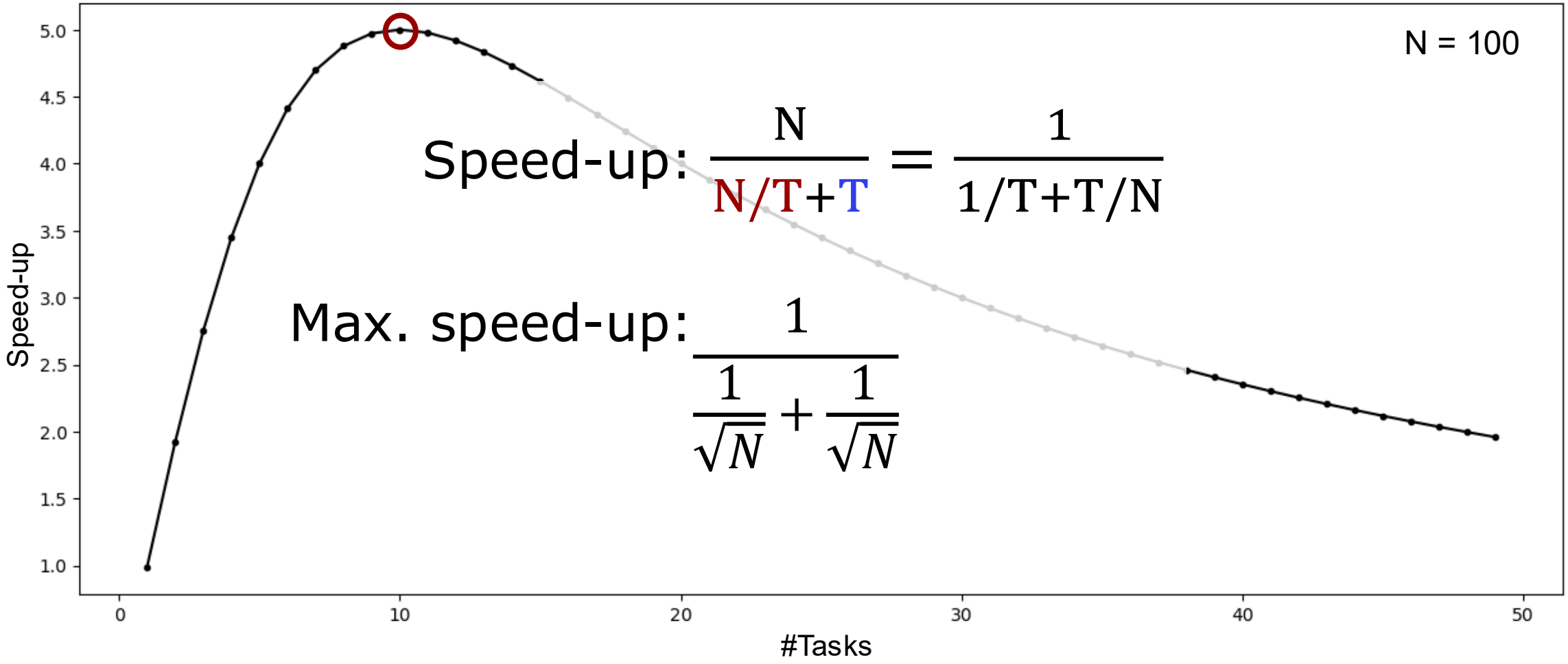
Motivation: parallel sum



Speed-up: $\frac{N}{N/T+T} = \frac{1}{1/T+T/N}$

Max. speed-up: $\frac{1}{\frac{1}{\sqrt{N}} + \frac{\sqrt{N}}{N}}$

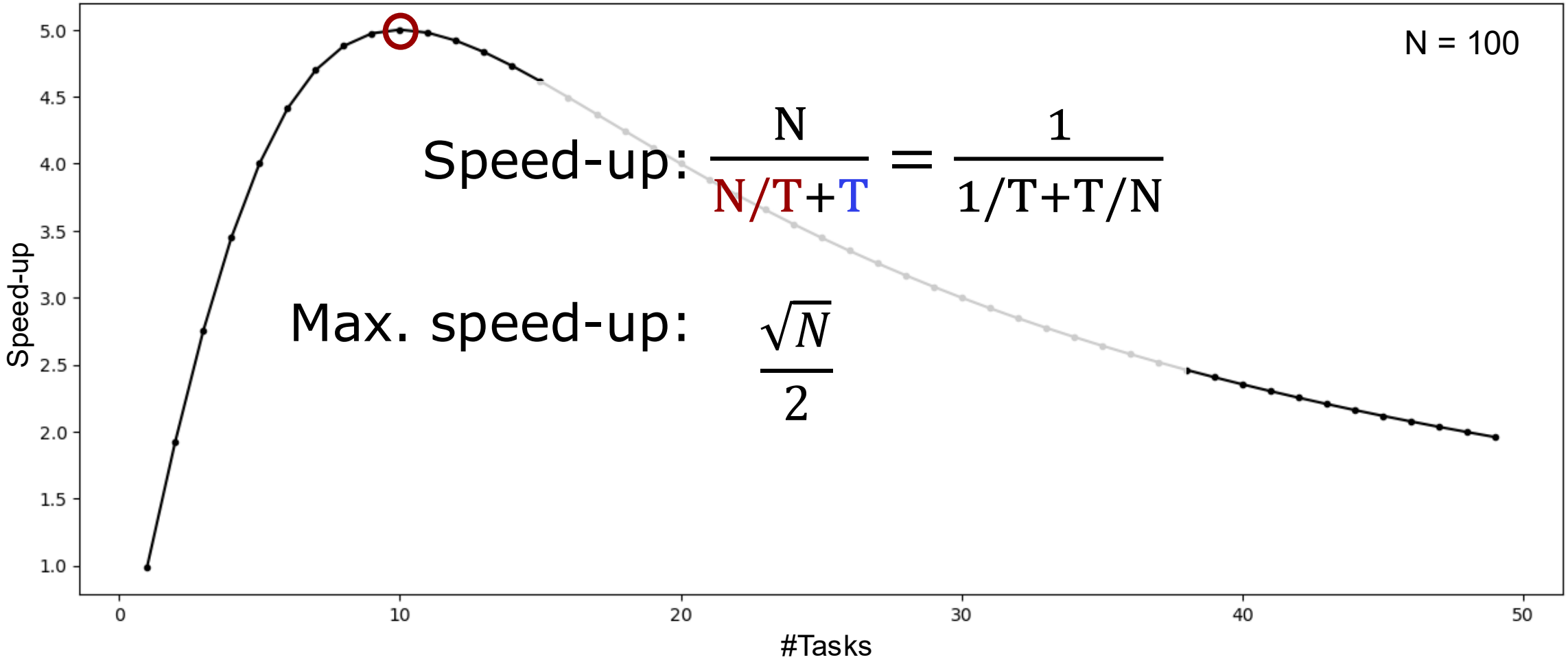
Motivation: parallel sum



Speed-up: $\frac{N}{N/T+T} = \frac{1}{1/T+T/N}$

Max. speed-up: $\frac{1}{\frac{1}{\sqrt{N}}+\frac{1}{\sqrt{N}}}$

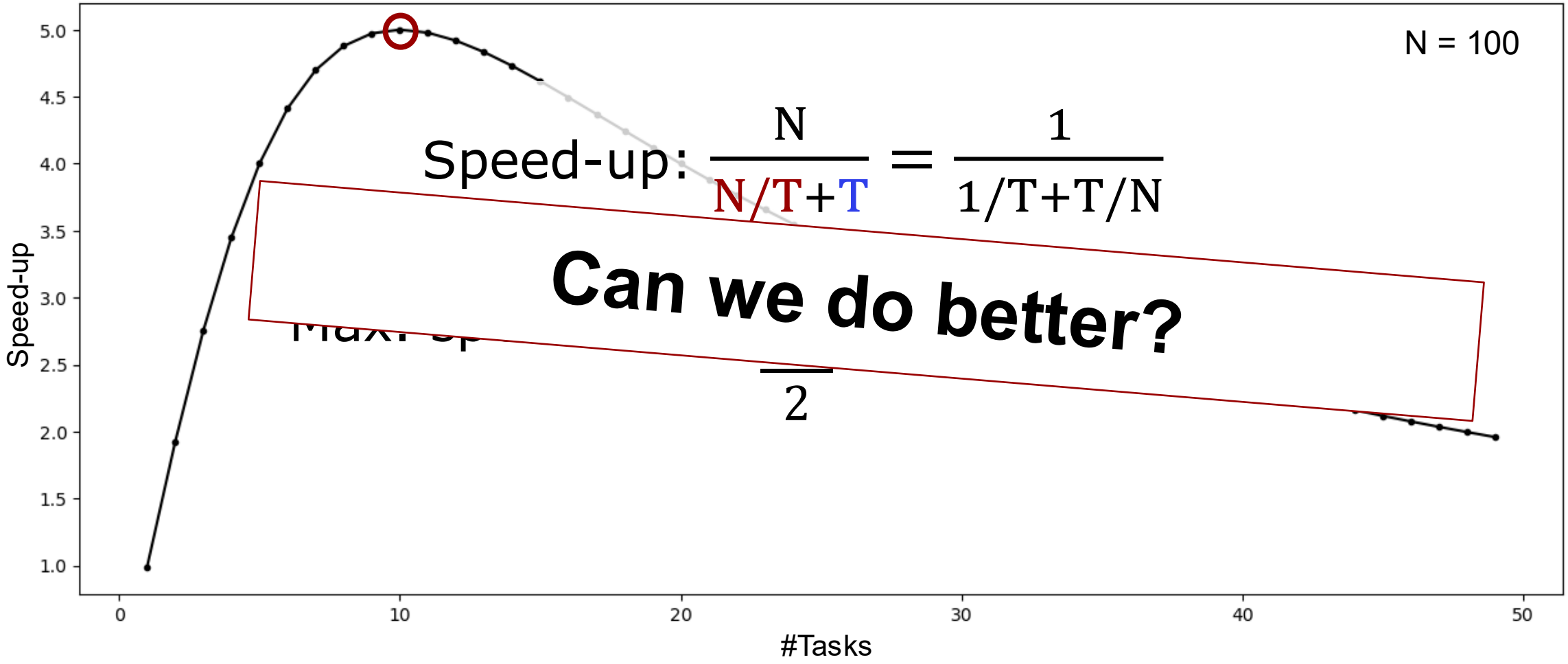
Motivation: parallel sum



Speed-up: $\frac{N}{N/T+T} = \frac{1}{1/T+T/N}$

Max. speed-up: $\frac{\sqrt{N}}{2}$

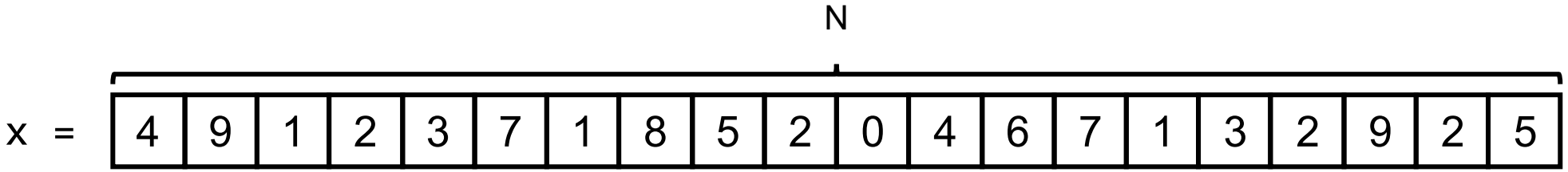
Motivation: parallel sum



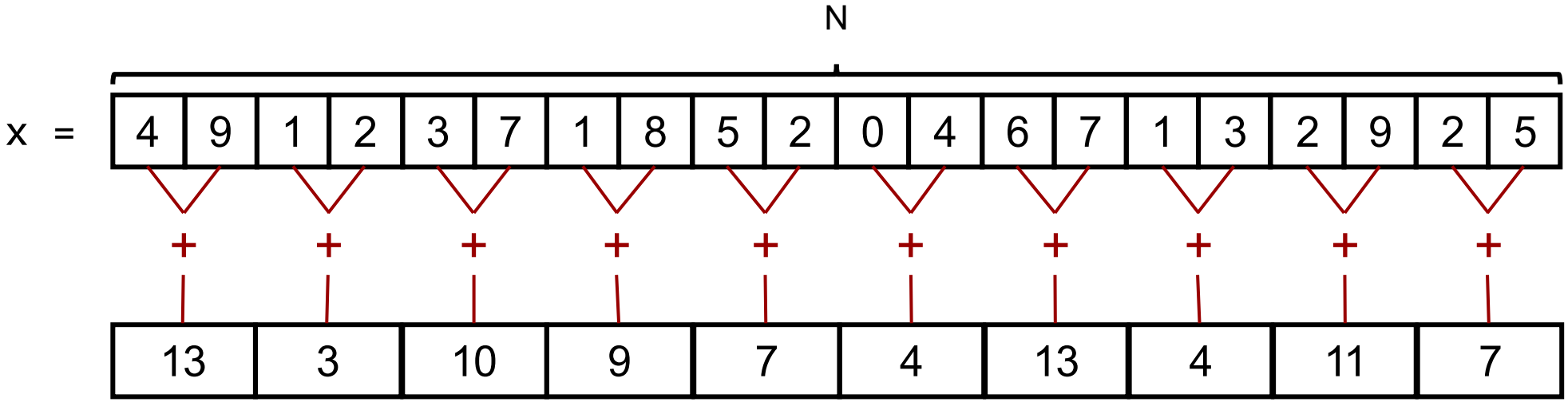
Speed-up: $\frac{N}{N/T+T} = \frac{1}{1/T+T/N}$

Can we do better?

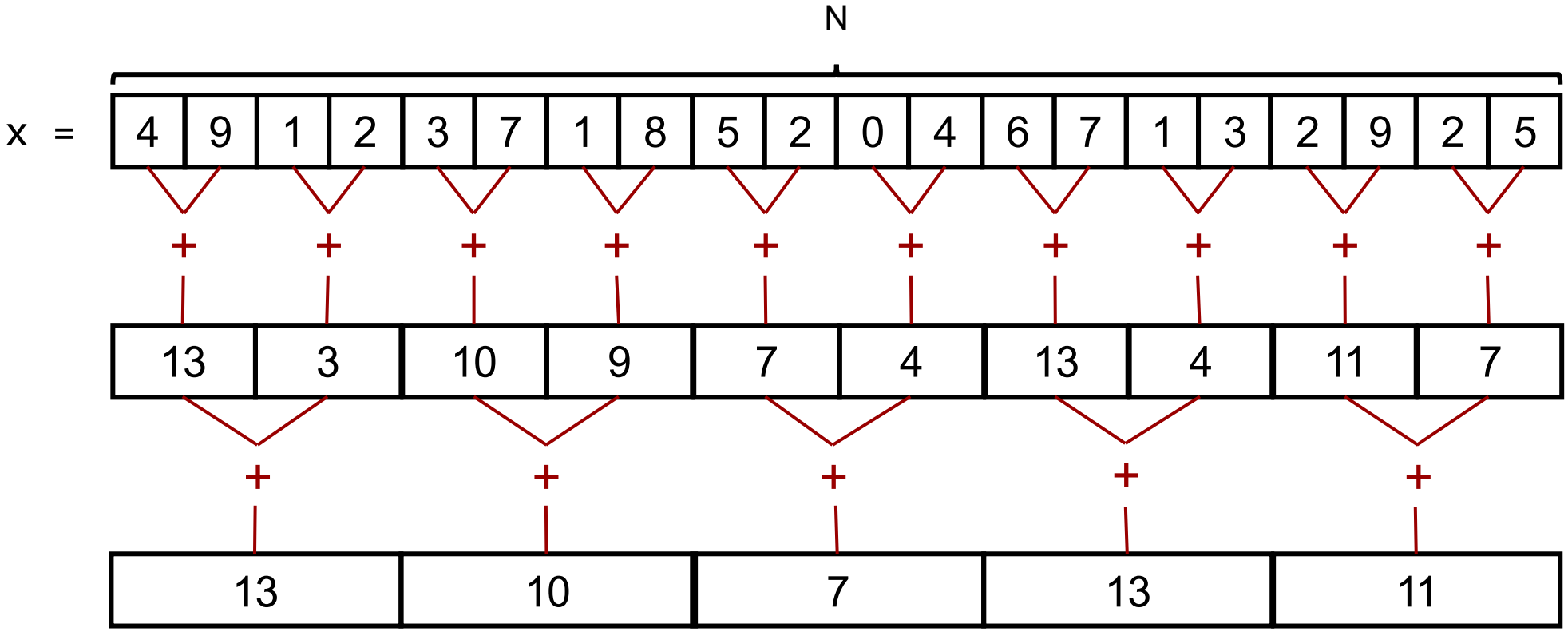
Reductions



Reductions

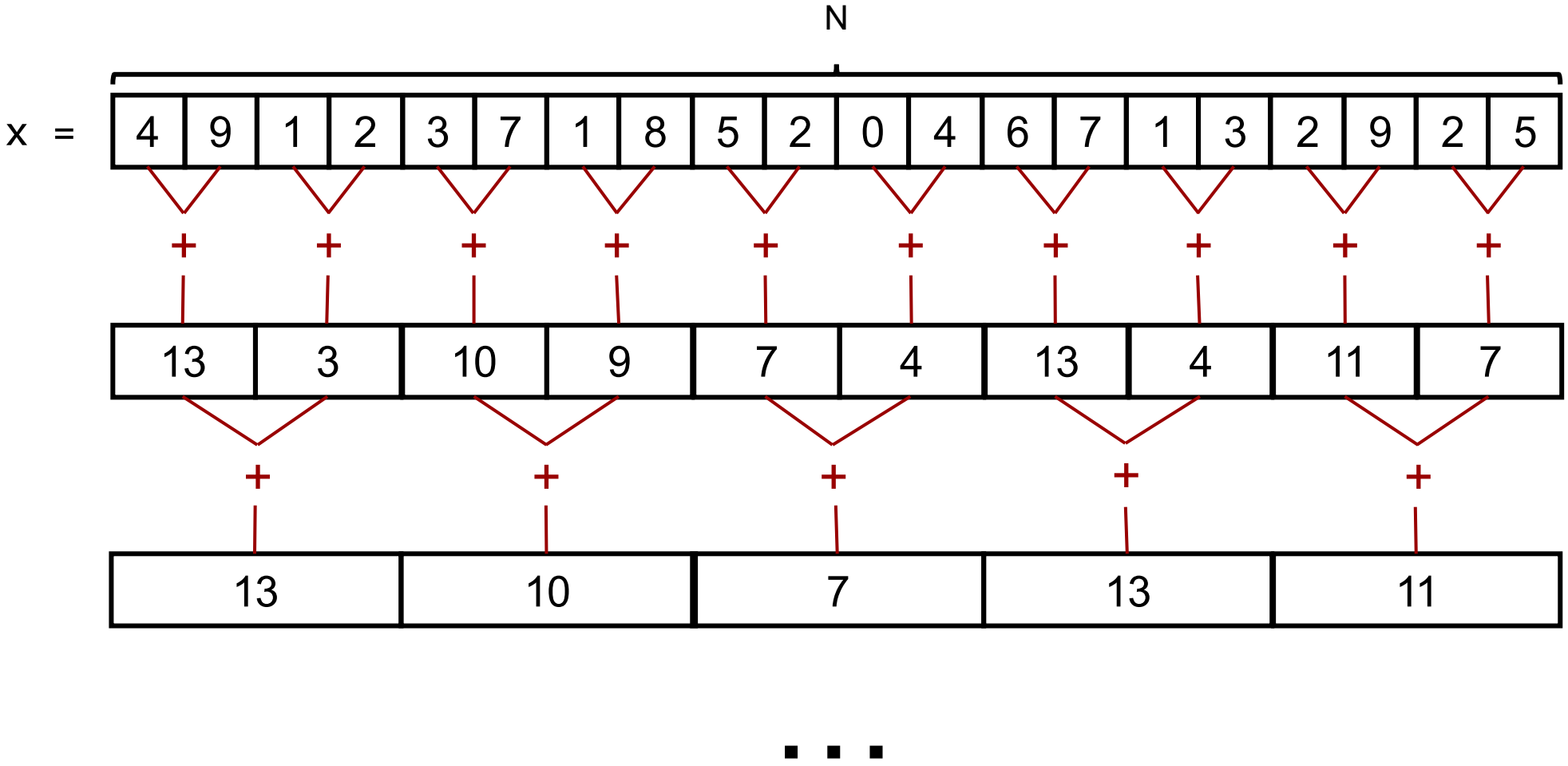


Reductions

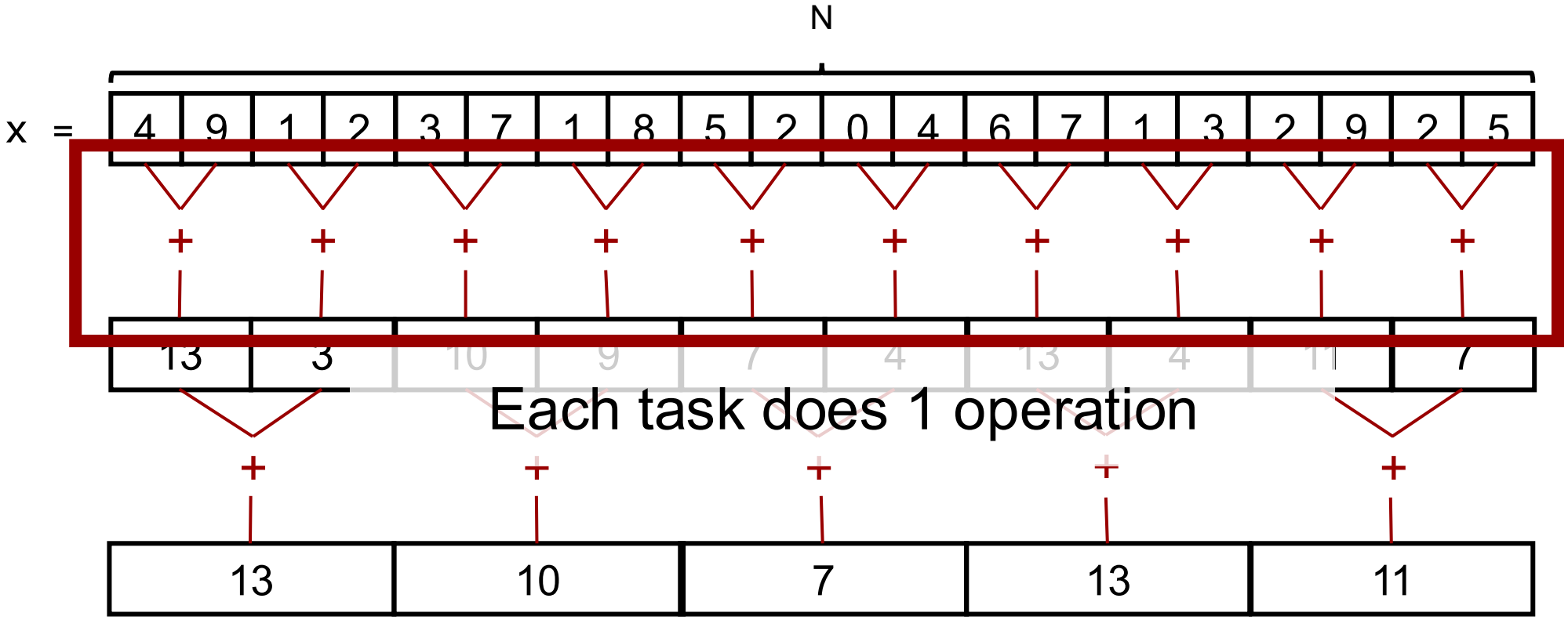


...

Binary tree reductions

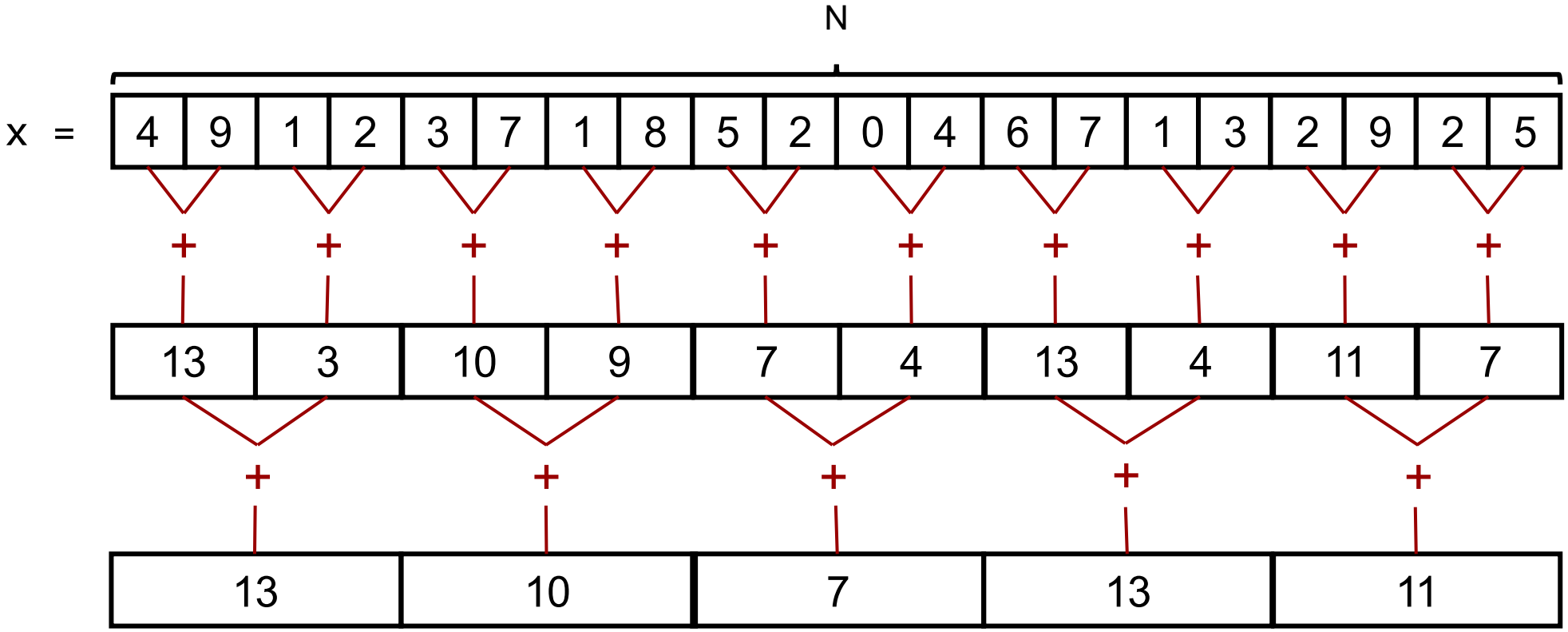


Binary tree reductions



...

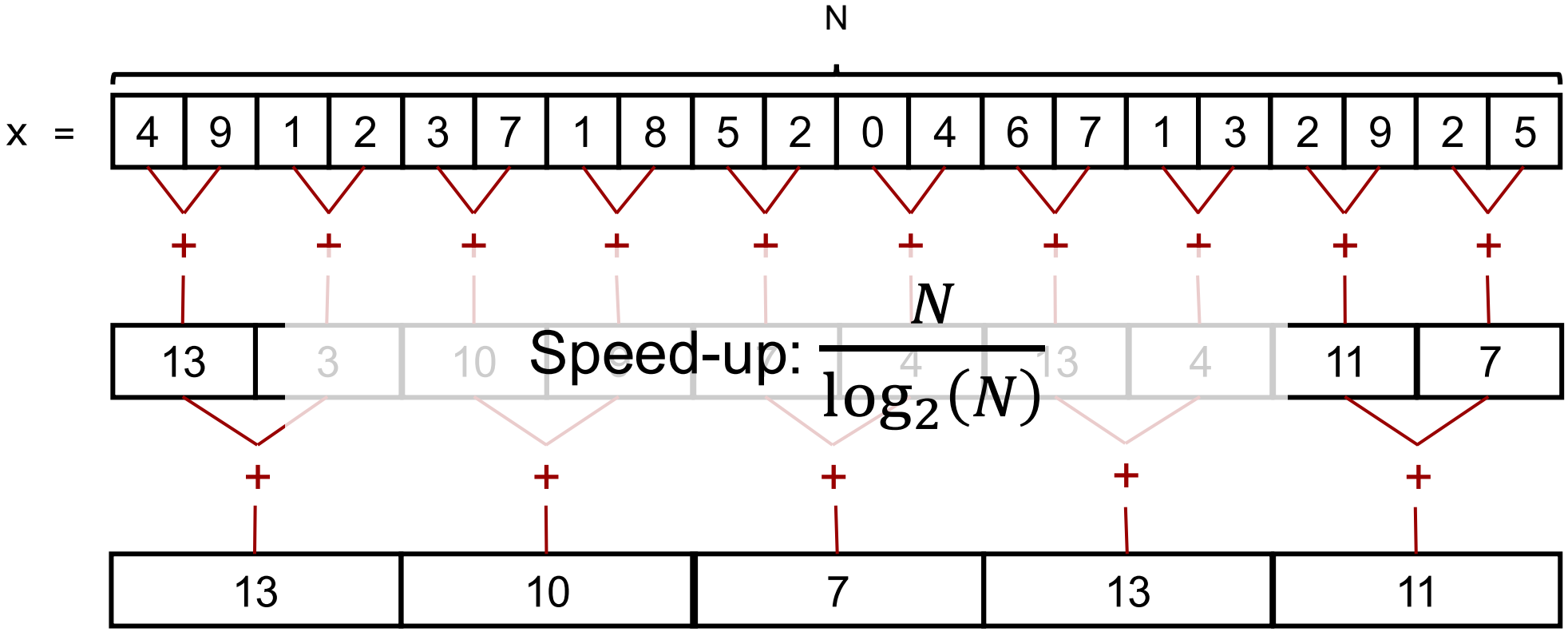
Binary tree reductions



Number of levels: $\log_2(\mathrm{N})$

...

Binary tree reductions

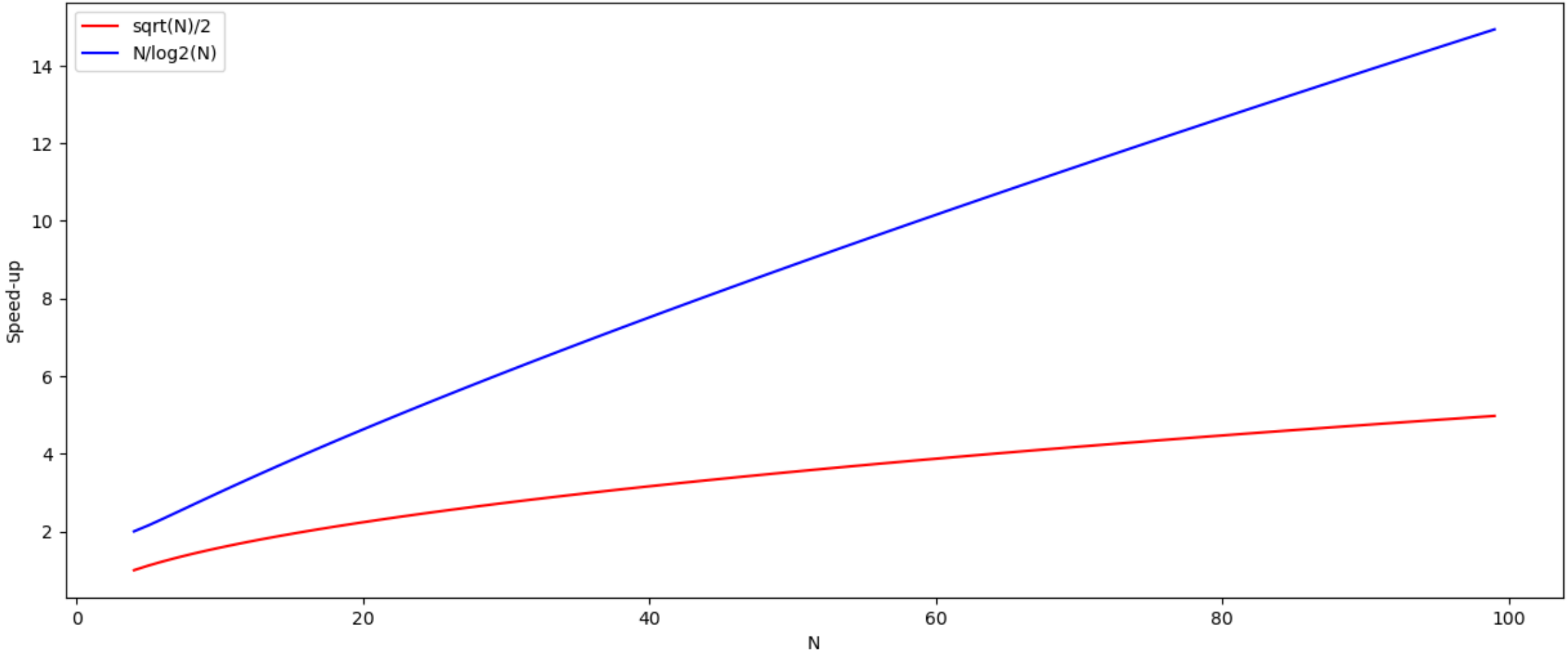


Speed-up: $\frac{N}{\log_2(N)}$

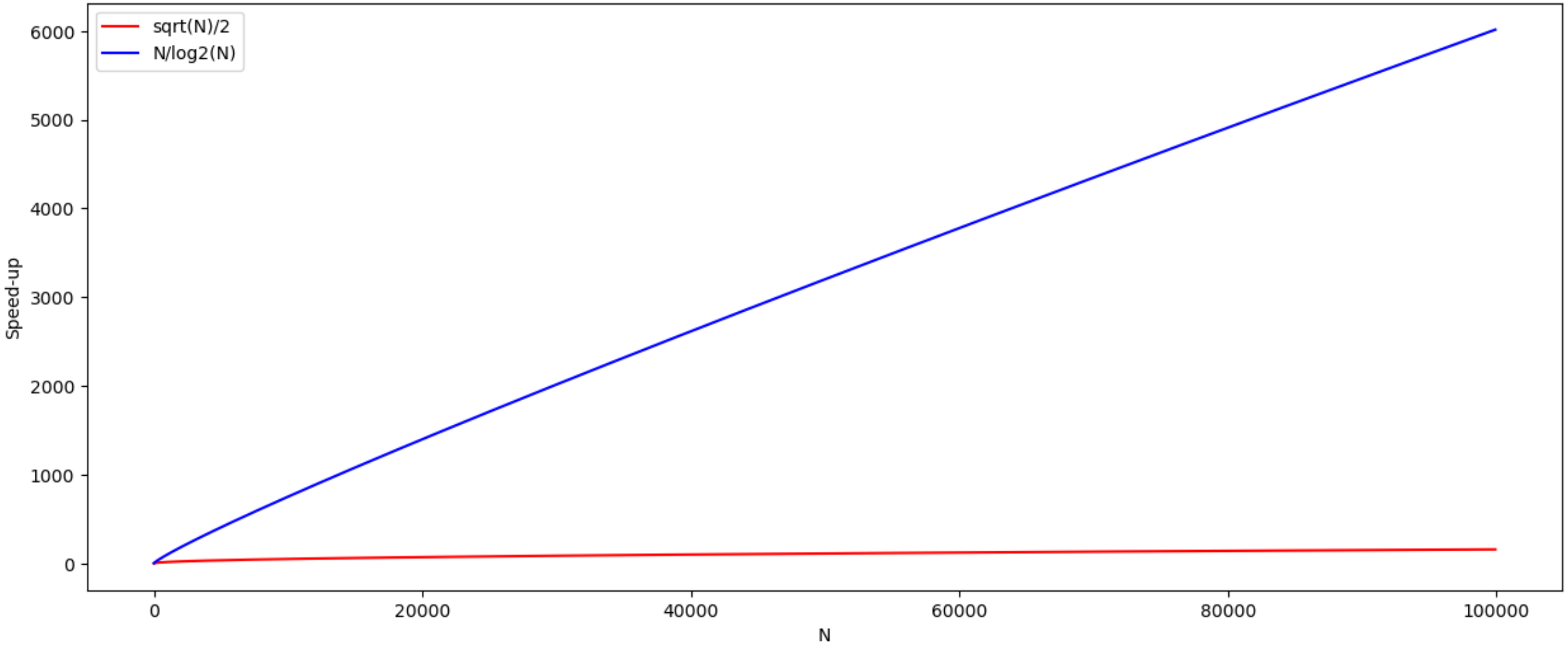
Number of levels: $\log_2(\mathrm{N})$

...

Reductions



Reductions


sqrt(N)/2
N/log2(N)
Speed-up
6000
5000
4000
3000
2000
1000
0
0
20000
40000
60000
80000
100000
N

Reductions

$$
\begin{aligned}
& 1 + 4 + 8 + 3 + 2 + 4 \\
= & (((((1 + 4) + 8) + 3) + 2) + 4) \\
= & ((1 + 4) + (8 + 3)) + (2 + 4) \\
= & ((5 + 11) + 6) \\
= & 16 + 6 \\
= & 22
\end{aligned}
$$

Reductions

$$
\begin{aligned}
& 1 * 4 * 8 * 3 * 2 * 4 \\
= & (((((1 * 4) * 8) * 3) * 2) * 4) \\
= & ((1 * 4) * (8 * 3)) * (2 * 4) \\
= & ((4 * 24) * 8) \\
= & 96 * 8 \\
= & 768
\end{aligned}
$$

Reductions

$$
\begin{aligned}
& 1 \,\&\, 1 \,\&\, 0 \,\&\, 1 \,\&\, 0 \,\&\, 0 \\
= & (((((1 \,\&\, 1) \,\&\, 0) \,\&\, 1) \,\&\, 0) \,\&\, 0) \\
= & ((1 \,\&\, 1) \,\&\, (0 \,\&\, 1)) \,\&\, (0 \,\&\, 0) \\
= & ((1 \,\&\, 0) \,\&\, 0) \\
= & 0 \,\&\, 0 \\
= & 0
\end{aligned}
$$

A reduction operator $\bigcirc$ must be:

1. Commutative: $a \bigcirc b = b \bigcirc a$
2. Associative: $(a \bigcirc b) \bigcirc c = a \bigcirc (b \bigcirc c)$

”Implementation detail”: May want to re-order inputs during processing

”Obvious”: Need to split into parallel tasks

Reductions

$$1 \wedge 4 \wedge 8 \wedge 3 \wedge 2 \wedge 4$$
$$= (((((1 \wedge 4) \wedge 8) \wedge 3) \wedge 2) \wedge 4)$$
$$\neq ((1 \wedge 4) \wedge (8 \wedge 3)) \wedge (2 \wedge 4)$$

A reduction operator $\bigcirc$ must be:

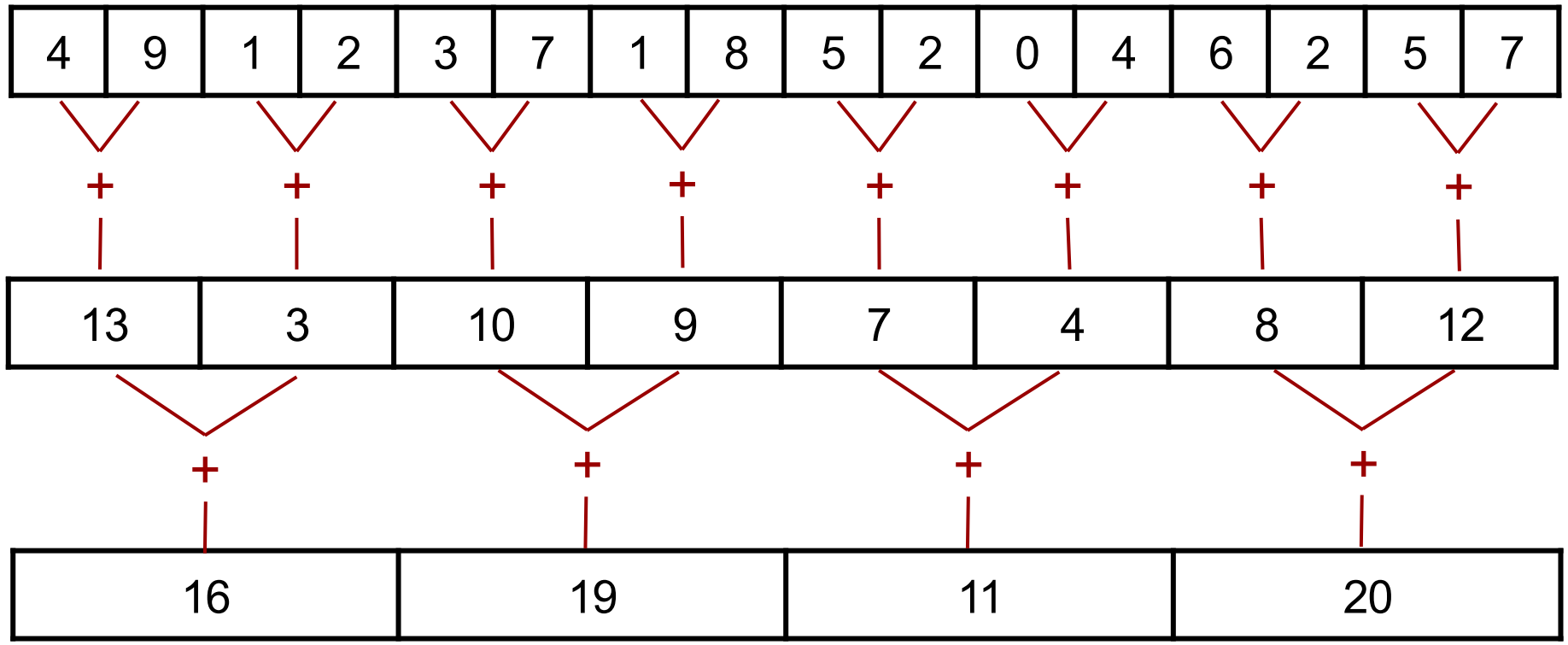
1. Commutative: $a \bigcirc b = b \bigcirc a$
2. Associative: $(a \bigcirc b) \bigcirc c = a \bigcirc (b \bigcirc c)$

Reductions

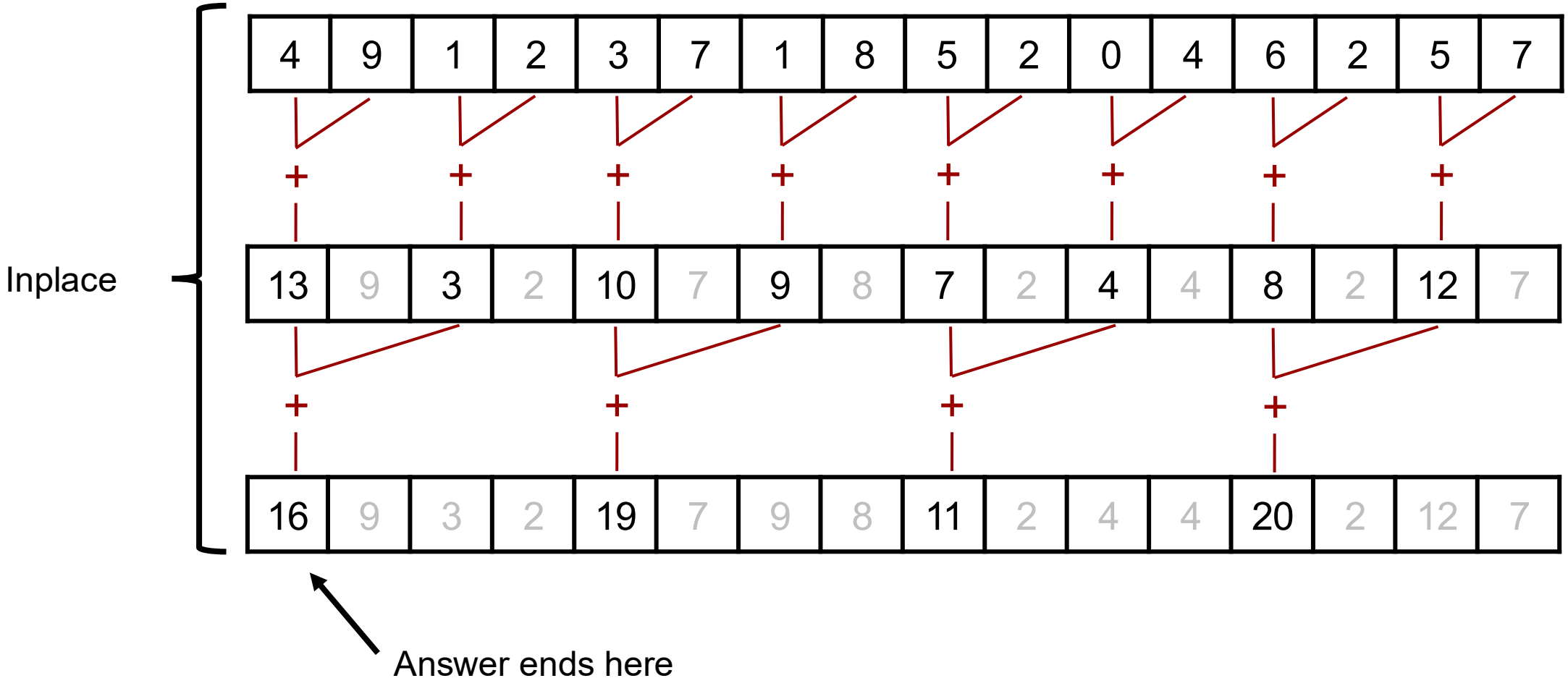
General framework: `reduce(operation, data)`

MapReduce:

Reductions in practice



Reductions in practice



Reductions in practice

```
s = 1  # Step
```

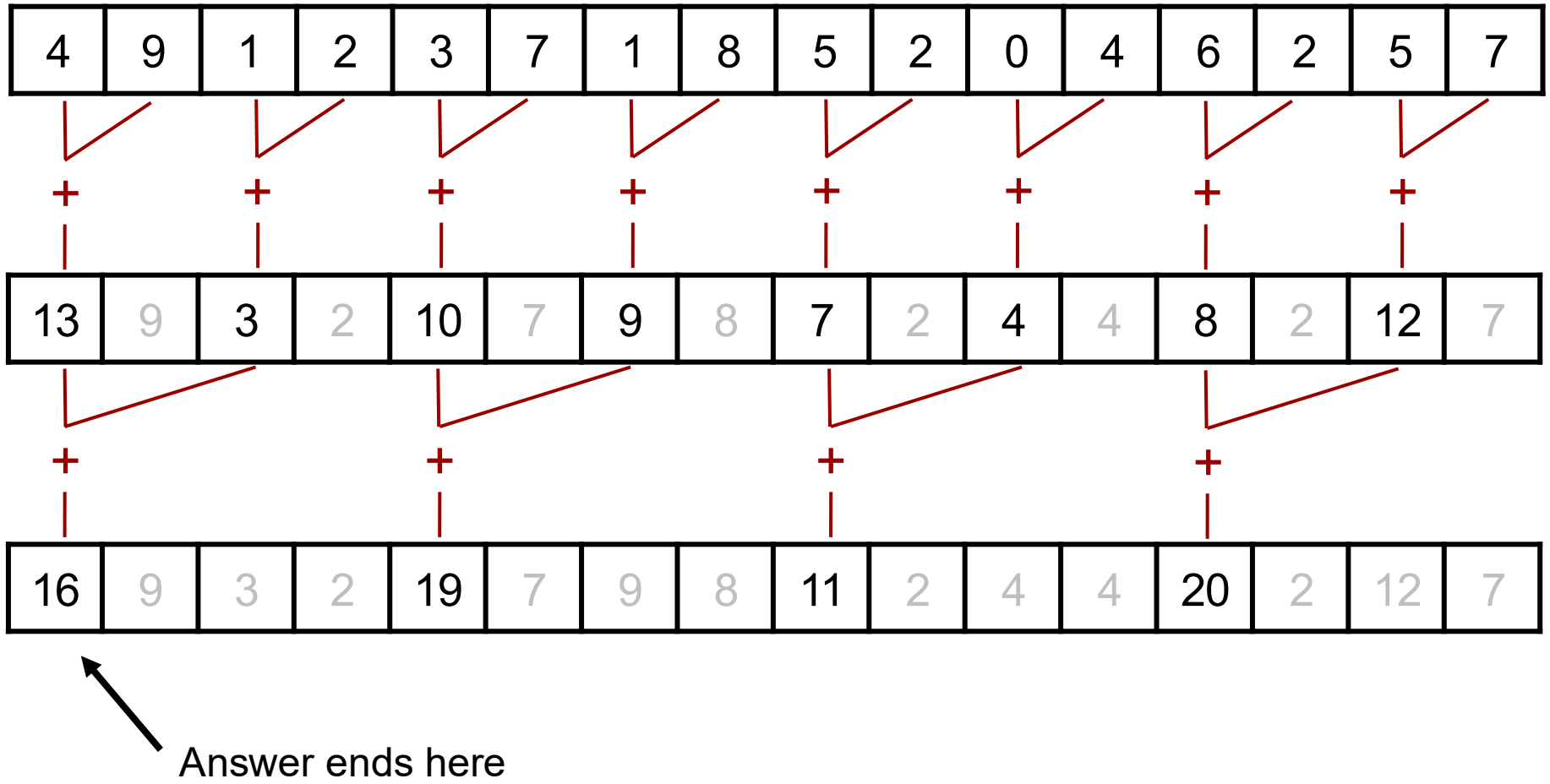
Parallel

```
for i in range(0,N,2*s):
    x[i] += x[i+s]
```

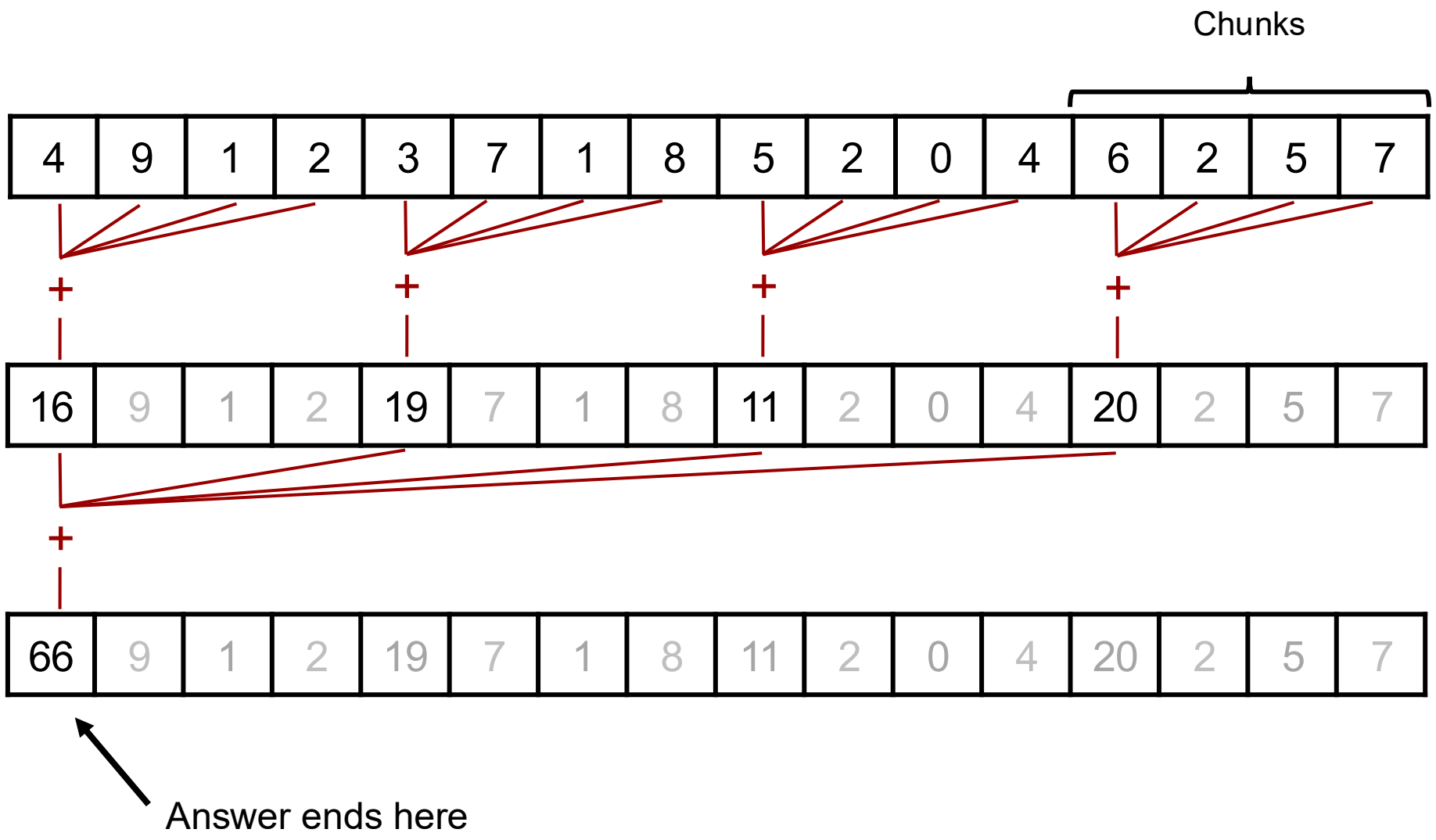
```
s = 2
```

Parallel

```
for i in range(0,N,2*s):
    x[i] += x[i+s]
```



Reductions in practice



Reductions in practice

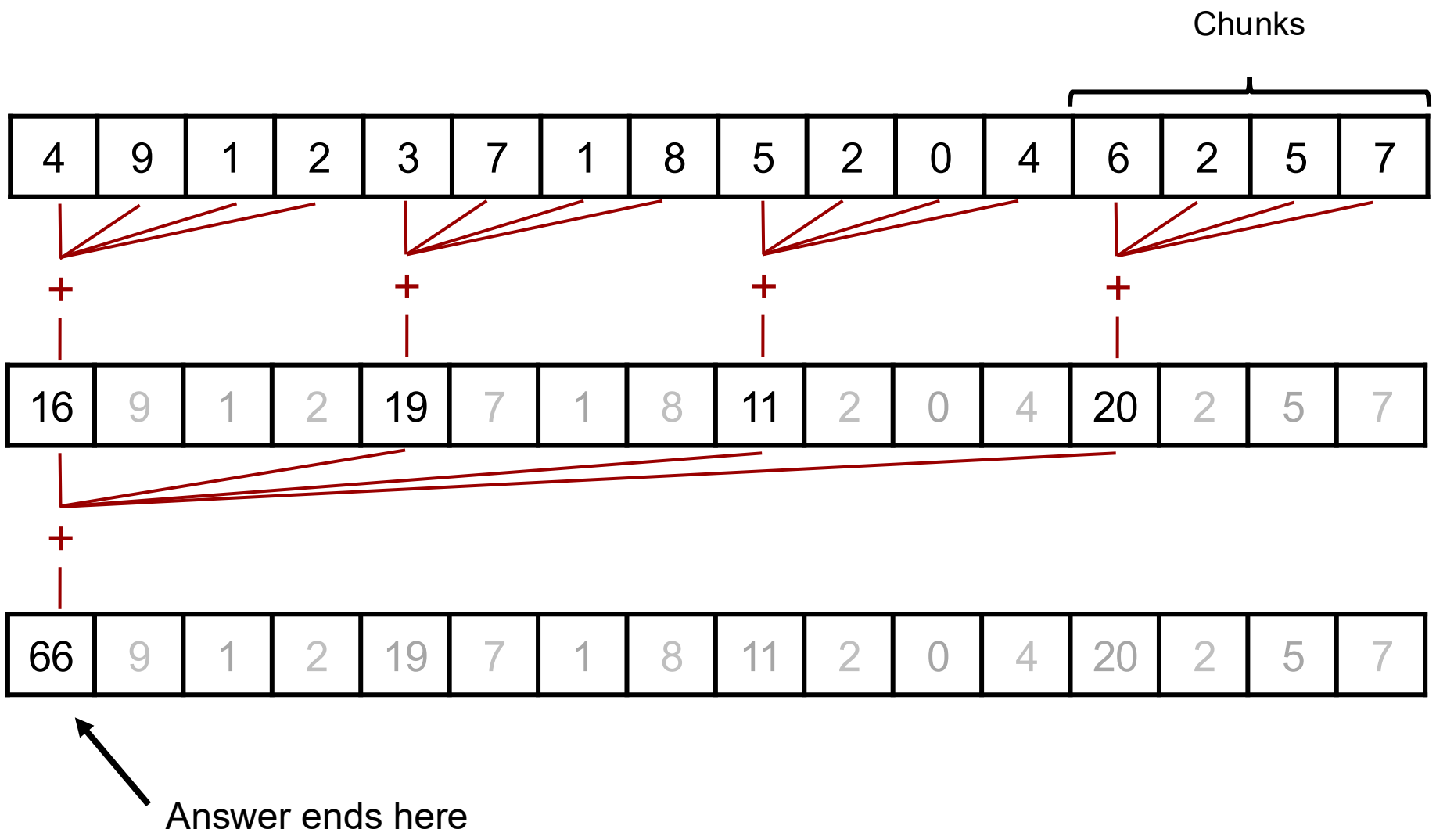
```
c = 4  # Chunk size

s = 1  # Step

for b in range(0,N,c*s):
    e = b + c * s
    x[b] = sum(x[b:e:s])

s = 4  # s = s * c

for b in range(0,N,c*s):
    e = b + c * s
    x[b] = sum(x[b:e:s])
```



Reductions in practice

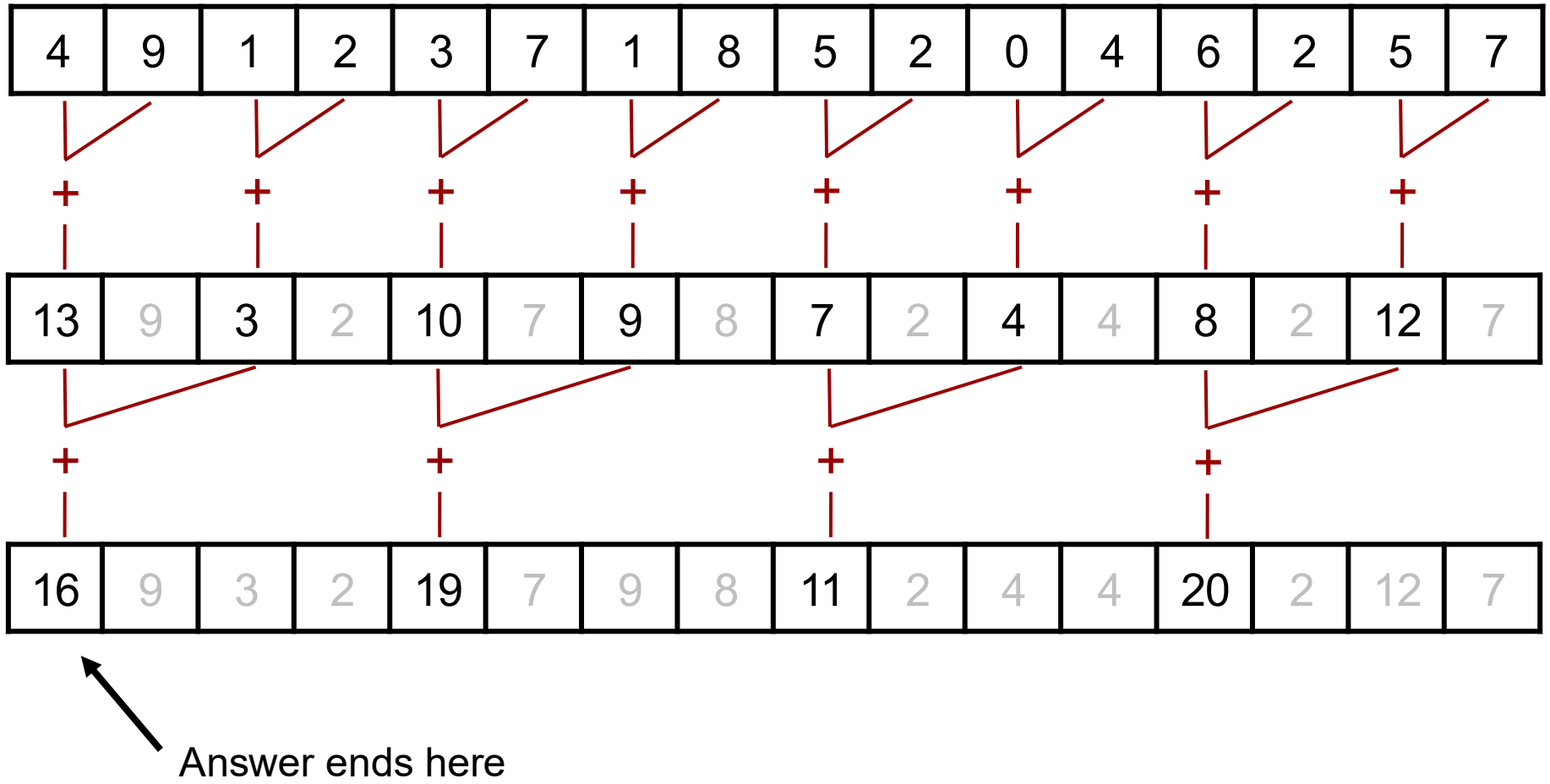
```
c = 2  # Chunk size

s = 1  # Step

for i in range(0,N,2*s):
    x[i] += x[i+s]

s = 2

for i in range(0,N,2*s):
    x[i] += x[i+s]
```



Reductions in practice

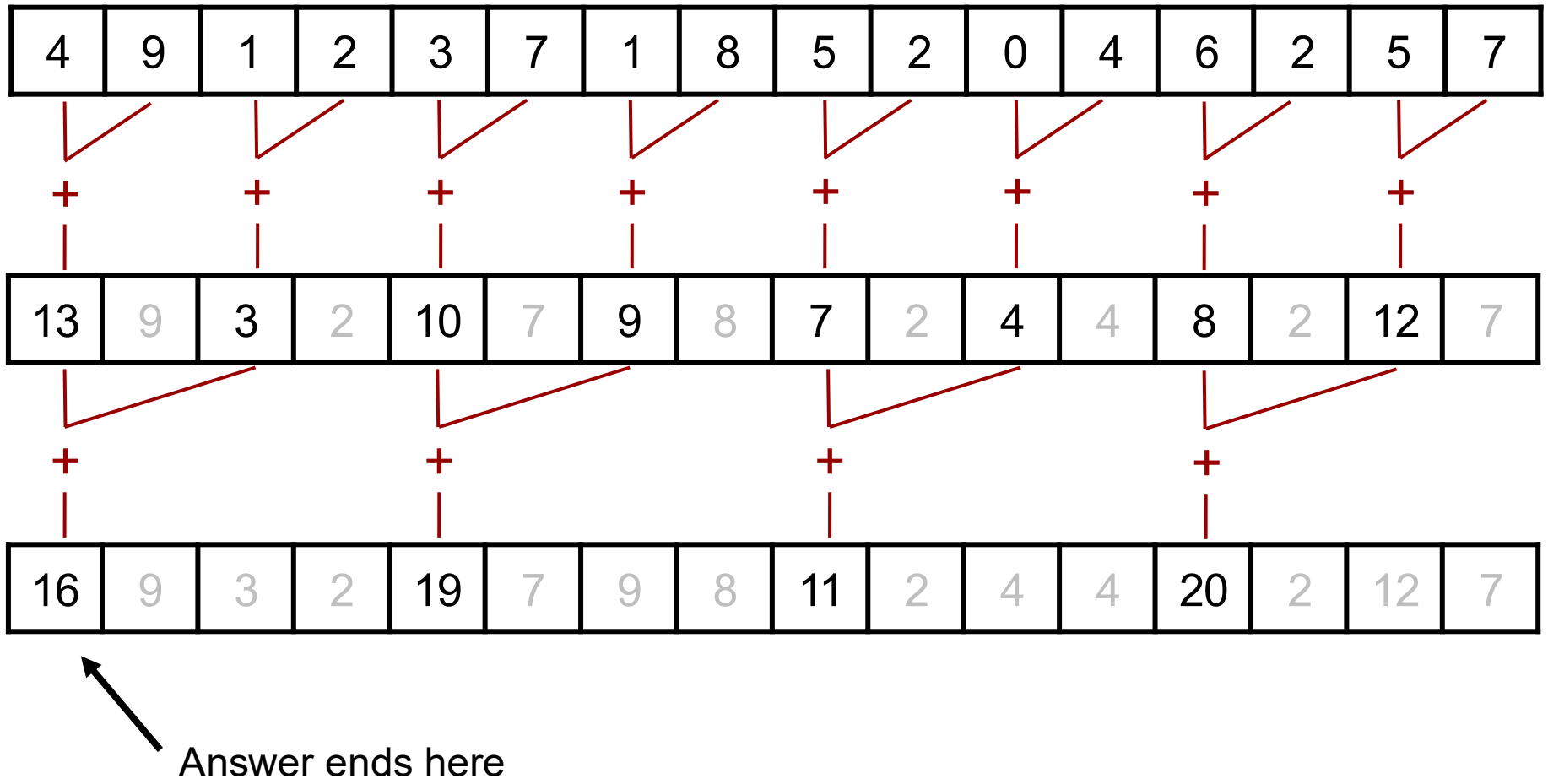
```
c = 2  # Chunk size

s = 1  # Step

for b in range(0,N,c*s):
    e = b + c * s
    x[b] = sum(x[b:e:s])

s = 2  # s = s * c

for b in range(0,N,c*s):
    e = b + c * s
    x[b] = sum(x[b:e:s])
```

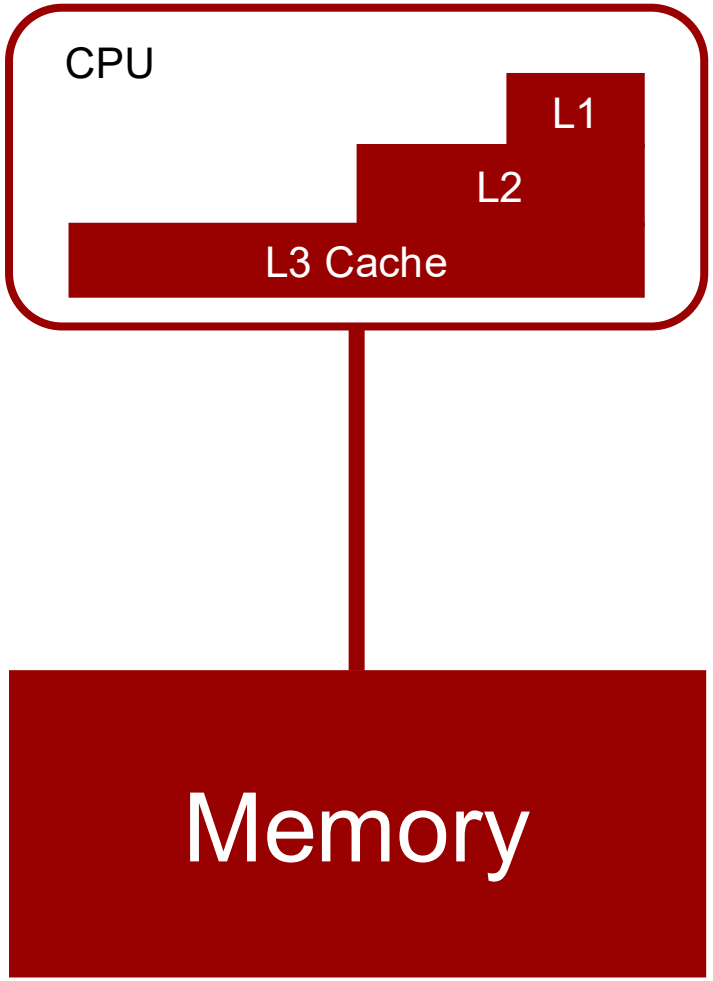


Quiz time!

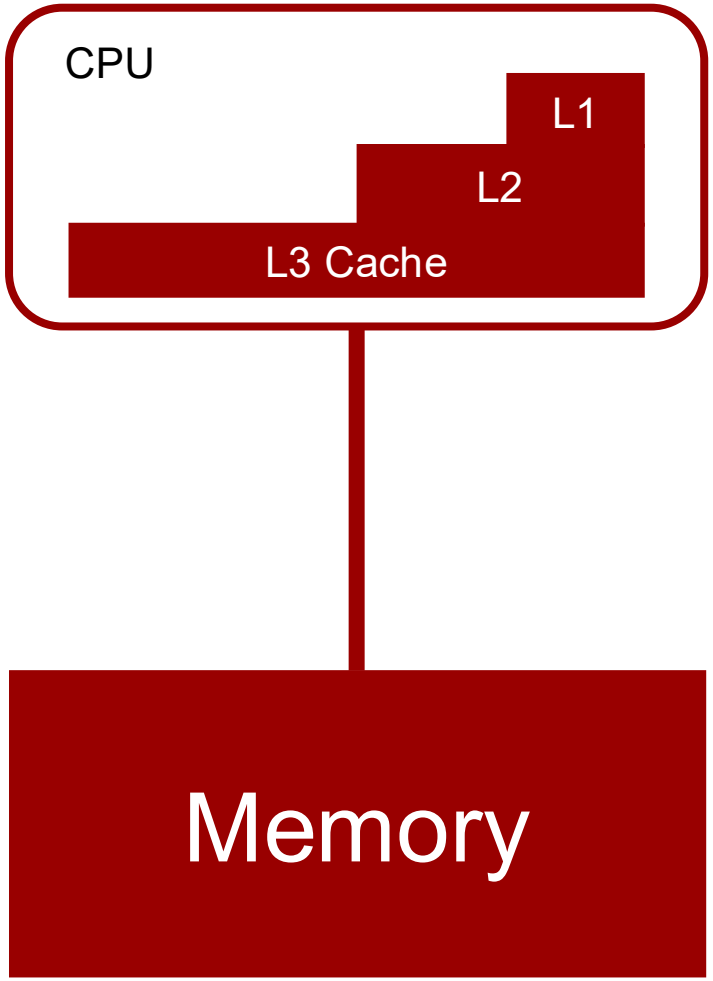
NUMA

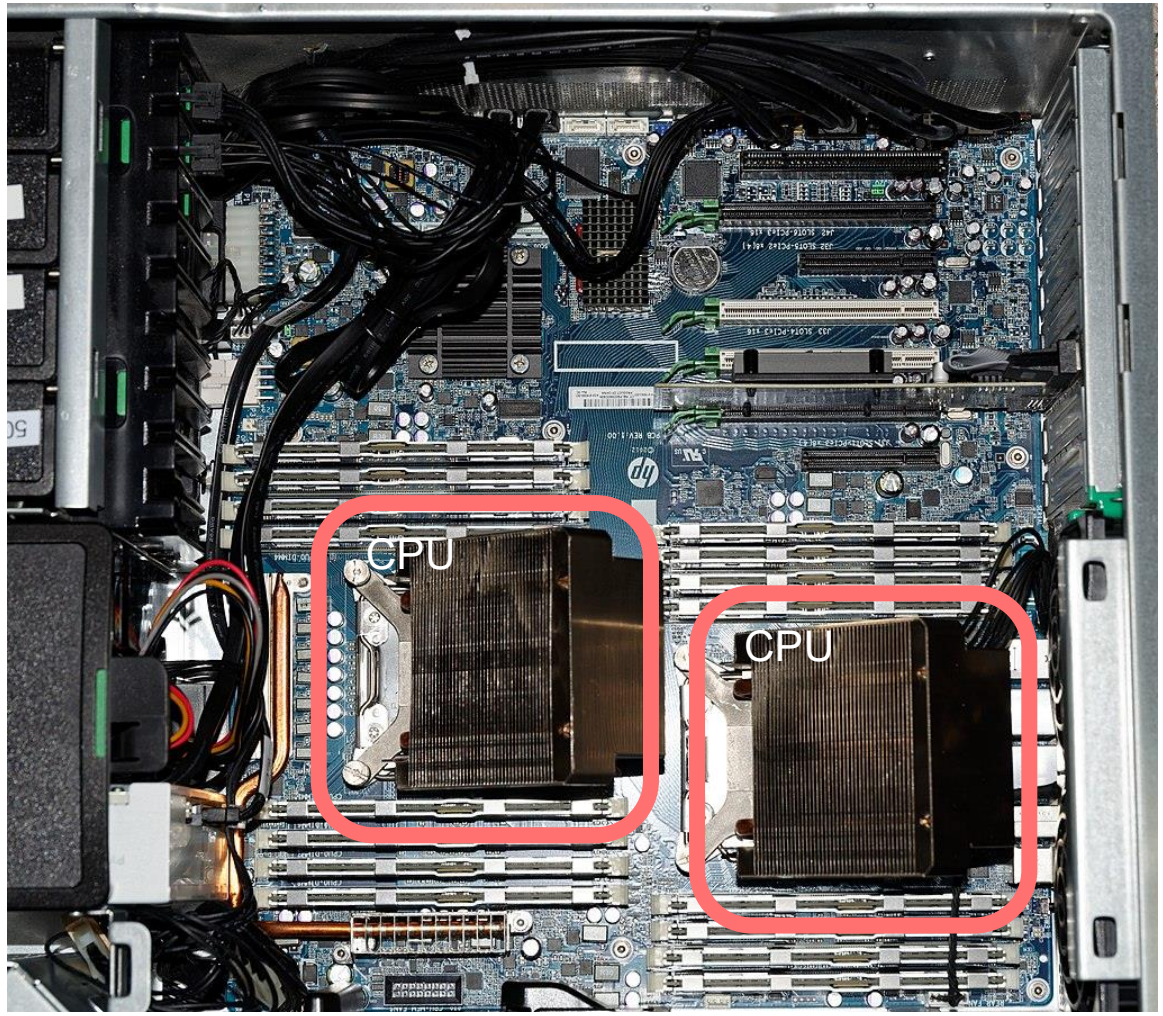
Non-**U**niform **M**emory **A**ccess

NUMA



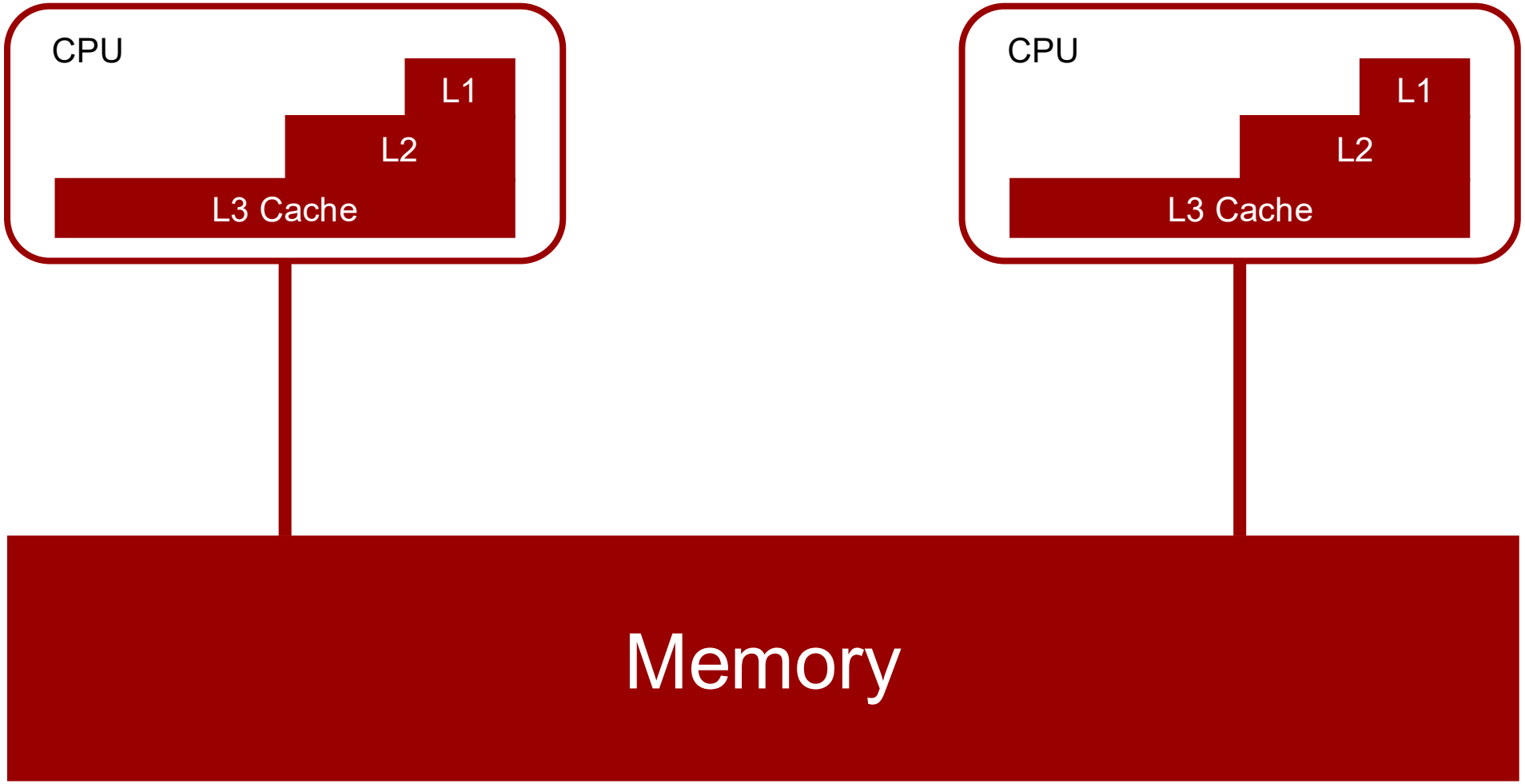
NUMA



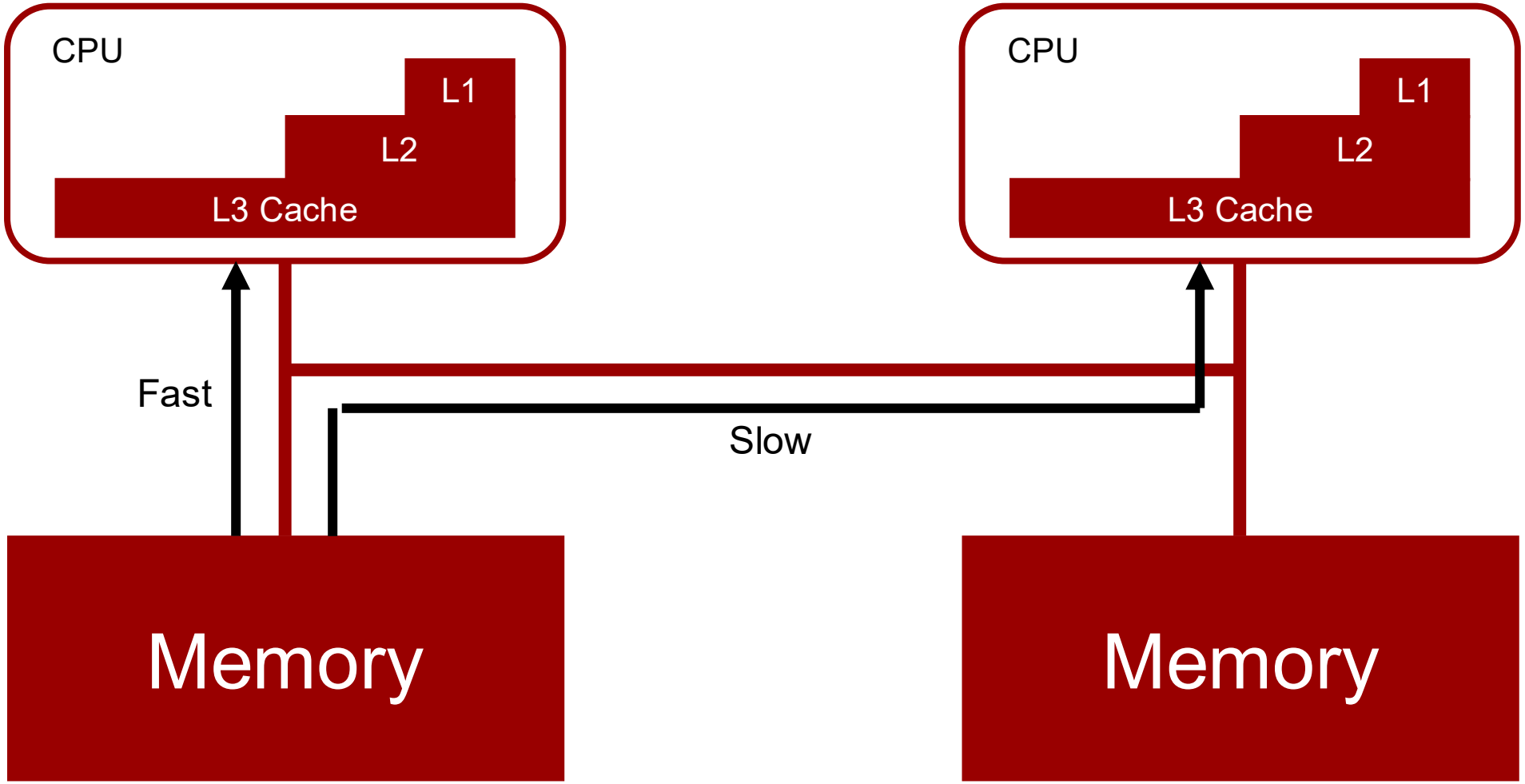


Jud McCranie, CC BY SA 4.0

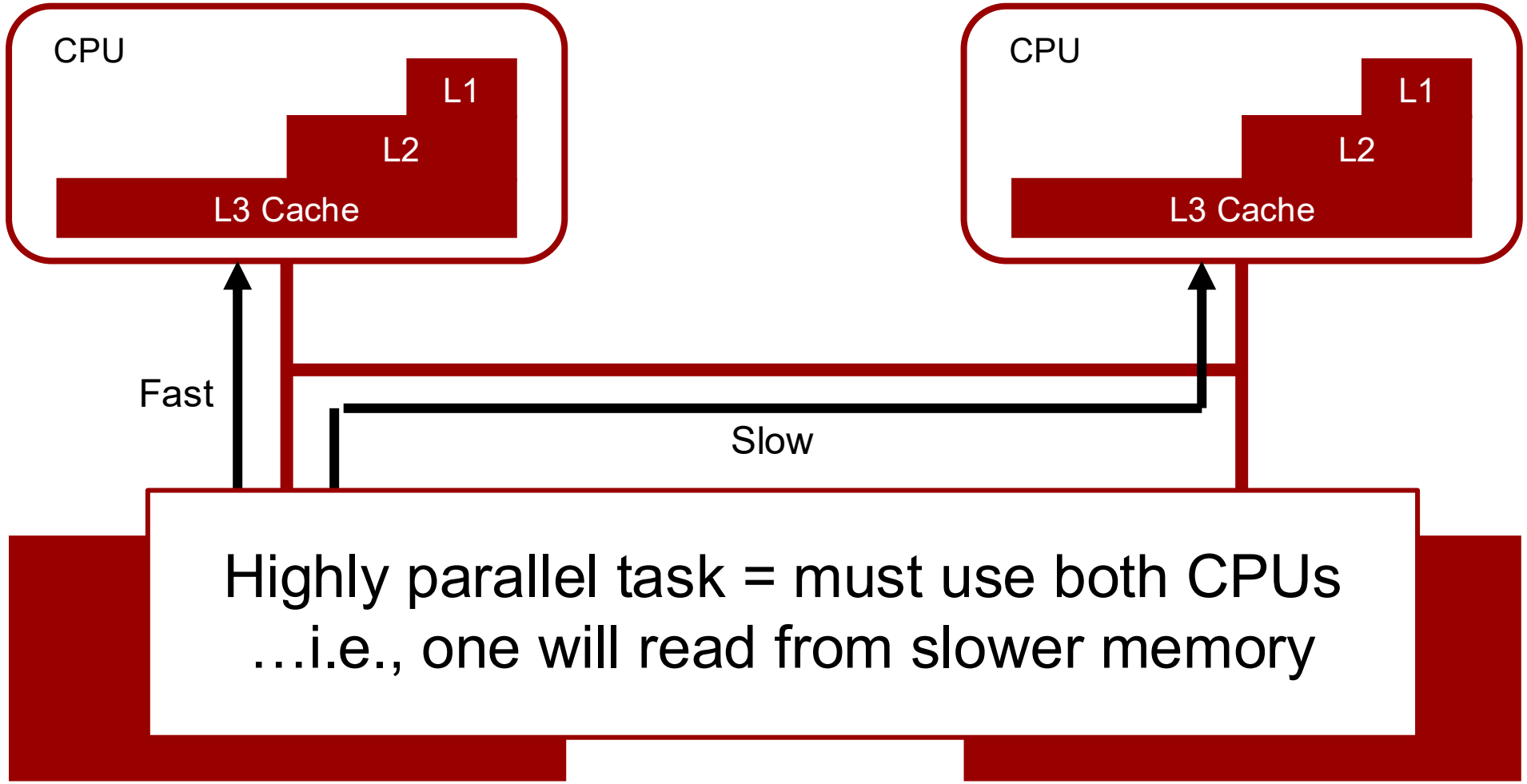
The fantasy



The reality: Non-Uniform Memory Access (NUMA)

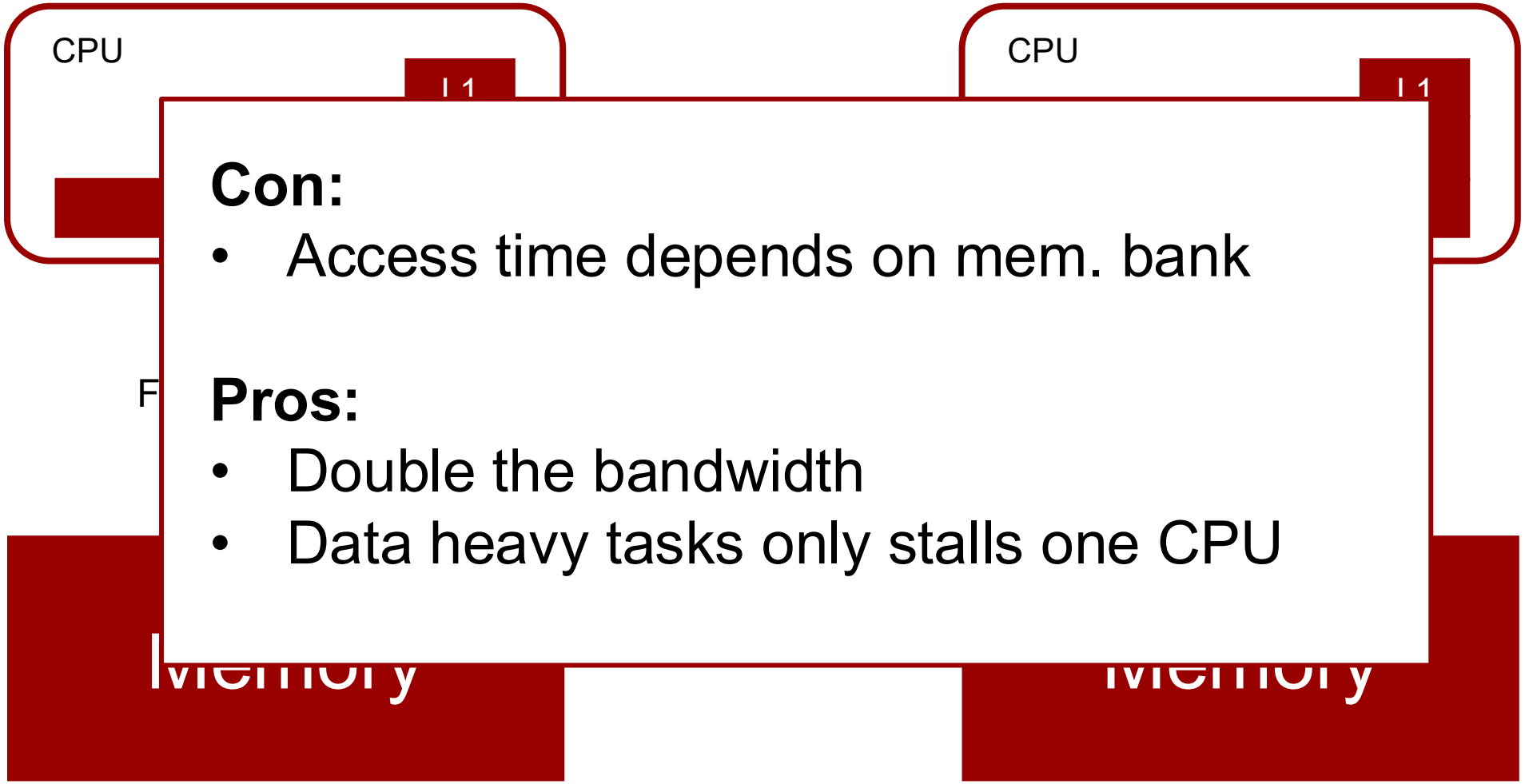


The reality: Non-Uniform Memory Access (NUMA)



Highly parallel task = must use both CPUs
…i.e., one will read from slower memory

The reality: Non-Uniform Memory Access (NUMA)



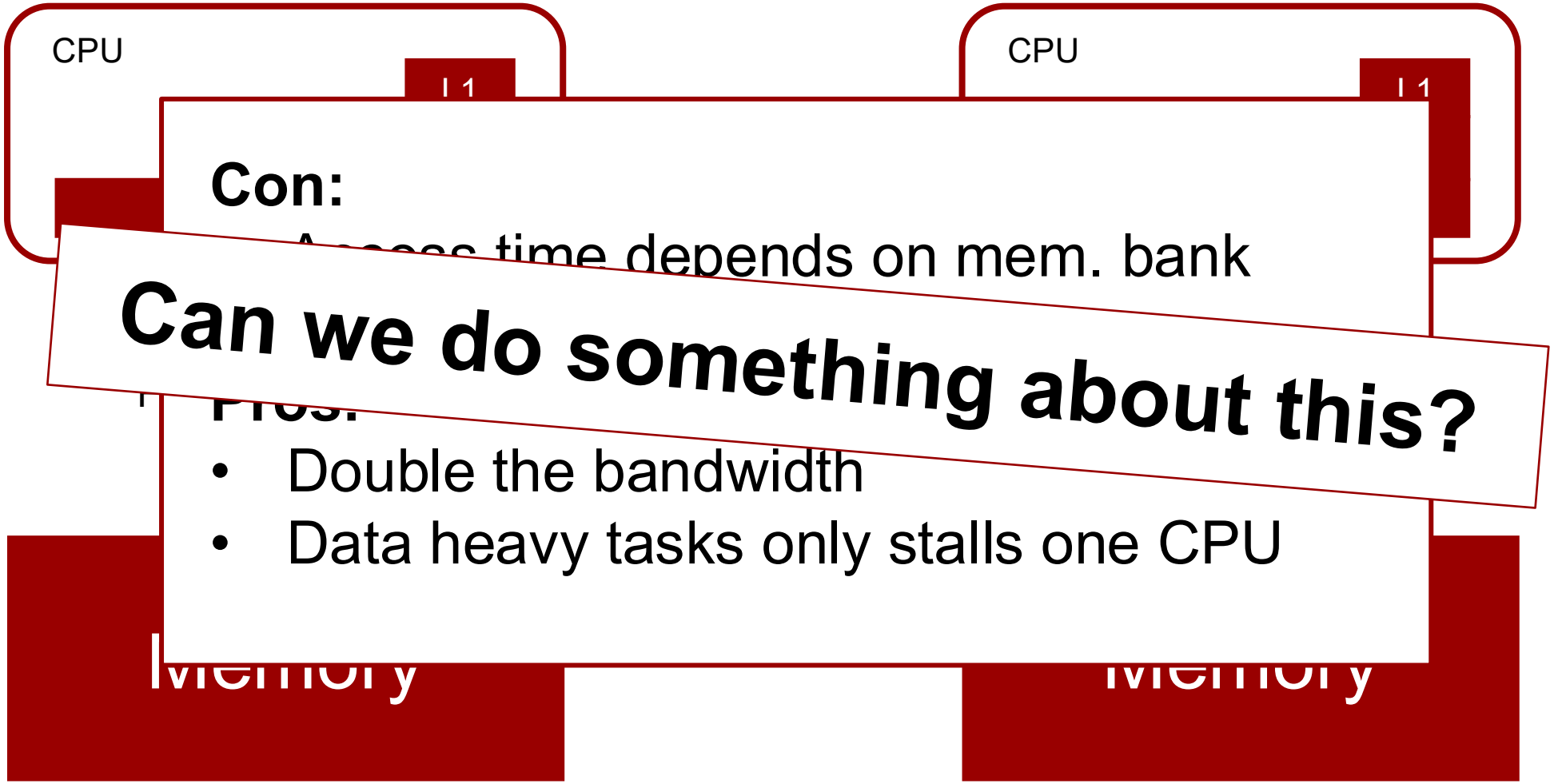
Con:

- Access time depends on mem. bank

Pros:

- Double the bandwidth
- Data heavy tasks only stalls one CPU

The reality: Non-Uniform Memory Access (NUMA)



Con:

- Access time depends on mem. bank

Can we do something about this?

Pros:

- Double the bandwidth
- Data heavy tasks only stalls one CPU

Yes, but…

Good solutions require low level control

- Easier from languages like C, C++, etc.
- See course 02614 High Performance Computing

Can play with it through command `numactl` (numa control)

- `numactl --interleave=all` (distribute on both mem. banks)

Yes, but…

Good solutions require low level control

- Easier from languages like C, C++, etc.
- See course 02614 High Performance Computing

Can play with it through command `numactl` (numa control)

- `numactl --interleave=all` (distribute on both mem. banks)

Yes, but…

Good solutio

- Easier fron
- See course

Note: This can solve <u>scaling</u> but **not** necessarily <u>speed</u>!

Can play with it through command `numactl` (numa control)

- `numactl --interleave=all` (distribute on both mem. banks)

Yes, but…

Good solutions require low level control

- Easier from languages like C, C++, etc.
- See course 02614 High Performance Computing

Can play with it through command `numactl` (numa control)

- `numactl --interleave=all` (distribute on both mem. banks)

- `numactl --cpunodebind=0` (only use cores on socket 0)
- `numactl --membind=0` (only allocated on mem. bank 0)
- See `man numactl` for more

Yes, but…

Good solutions require low level control

- Easier from languages like C, C++, etc.
- [illegible] Computing
- numactl [illegible]

Why care?
Because this is becoming normal!

- `numactl --cpunodebind=0` (only use cores on socket 0)
- `numactl --membind=0` (only allocated on mem. bank 0)
- See `man numactl` for more

Quiz time!

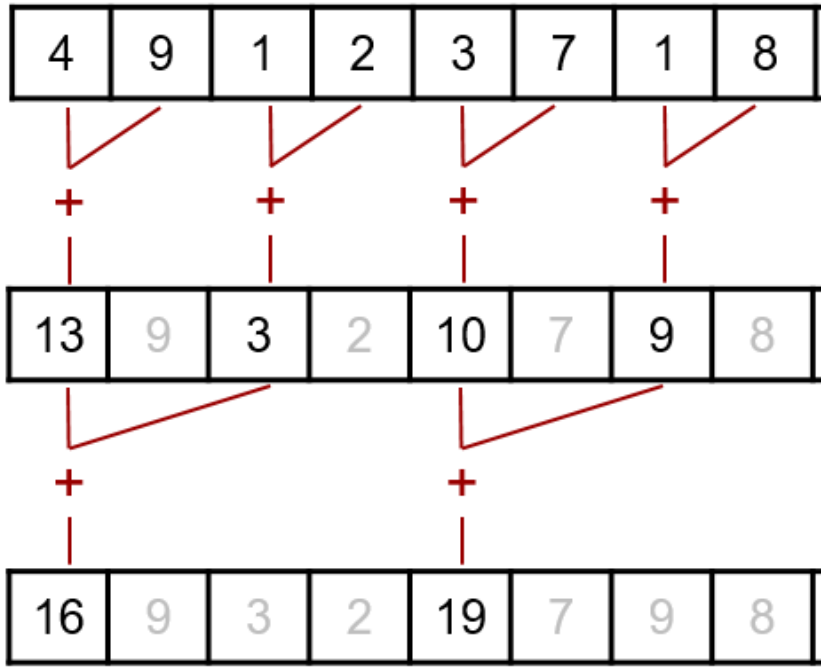
Today’s exercise

Implement a parallel tree reduction

- Finish last week's exercises
- Make a reduction over images
- Observe effect of NUMA

Useful concepts

Tree reduction



NUMA Control

```
numactl --interleave=all python script.py
```

Change to work node
linuxsh

Submit job script
bsub < submit.sh

Job status
bstat / bjobs

Check job output
bpeek / bpeek <JOBID>

Kill job
bkill <JOBID>

Time command for Python script
time python script.py